CYPHER Training School Scientific Machine Learning for Digital Twins

System Identification and machine learning for predicting reacting flows dynamics

https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026

Anh Khoa Doan
Department of Aeronautics
n.doan@imperial.ac.uk

IMPERIAL

CYPHER

Acknowledgement

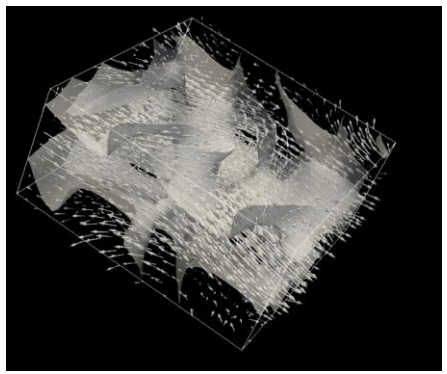
- Content heavily inspired by works/presentation of collaborators
  - Prof. Luca Magri (Imperial College):
    - Prediction of chaotic dynamics from data: An introduction (from the VKI lecture series) https://arxiv.org/pdf/2604.11624
    - https://github.com/MagriLab/Tutorials
  - Prof. Wolfgang Polifke (TU Munich):
    - Black-box system identification for reduced order model construction, *Annals of Nuclear Energy*, 67, 109-128 (2014). https://doi.org/10.1016/j.anucene.2013.10.037
    - https://gitlab.lrz.de/tfd/system-identification-tutorial
  - Content of the lecture: https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026

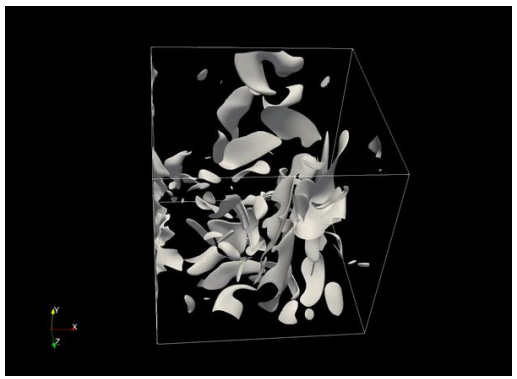
Outline

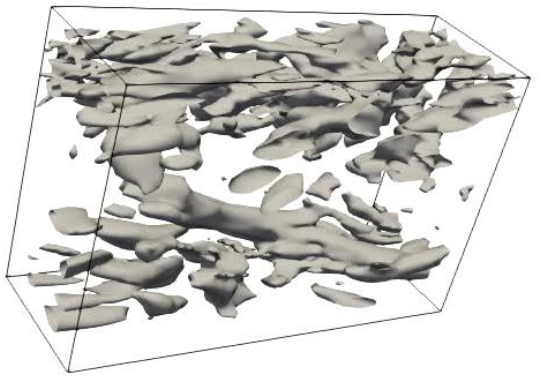
- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - System Identification for LTI systems
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

Turbulent (reacting) fluid flows are difficult to predict

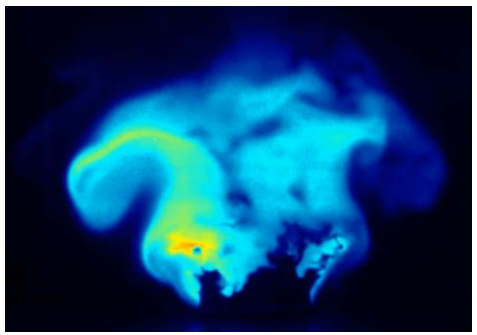
- Turbulence: multiscale, unsteadiness, uncertain, nonlinear
  → Unpredictable

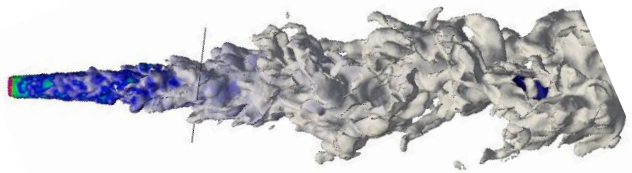






Added chemistry complexity in reacting flows





Dynamics of turbulent (reacting) flows

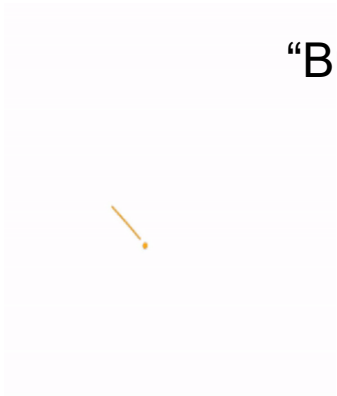
Nonlinear autonomous dynamical system:

$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$
$$\boldsymbol{\phi}(\boldsymbol{x}, 0) = \boldsymbol{\phi_0}$$

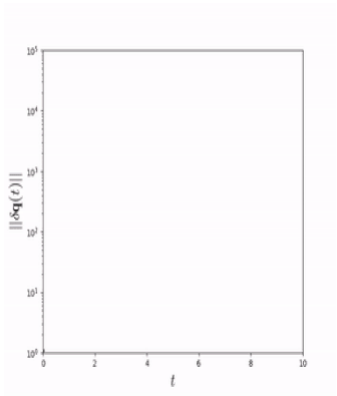
$\boldsymbol{N}$: smooth, deterministic and ergodic
$\boldsymbol{p}$: parameters of the systems
$\boldsymbol{F}$: (possible) forcing term

For fluids: $\boldsymbol{N}$: (Reacting) Navier-Stokes equations

Many systems of interest are *chaotic*



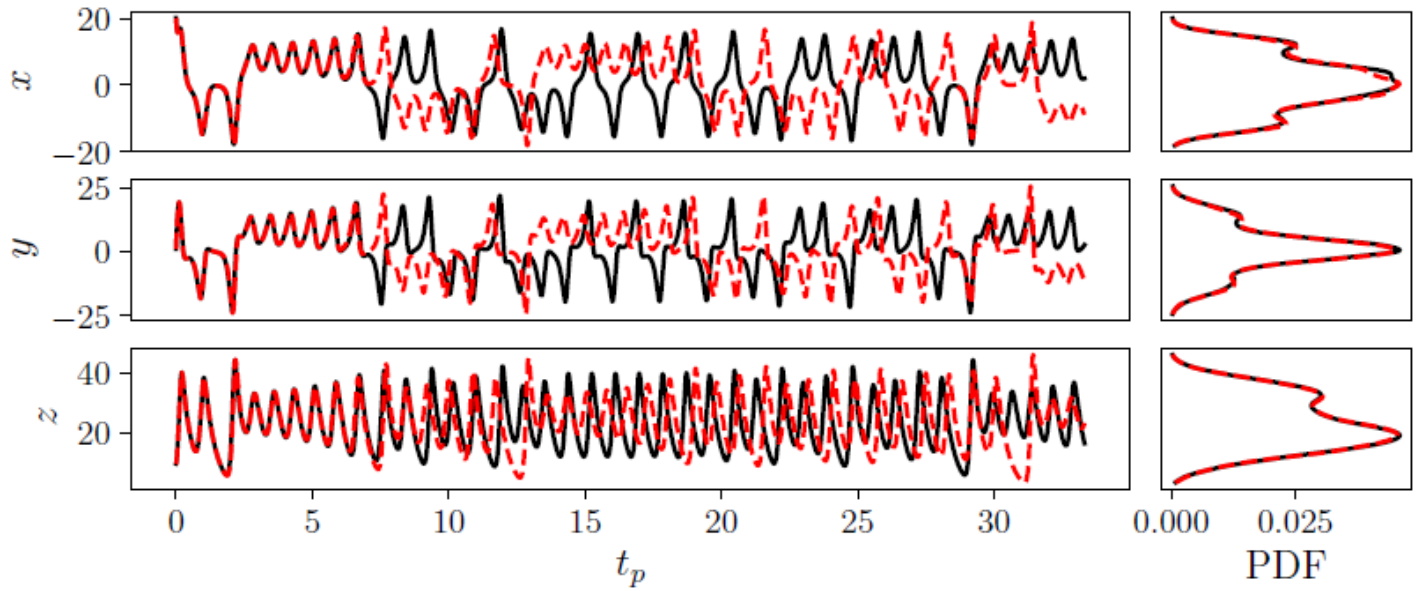
"Butterfly effect"



So what can/should we predict?

Statistics of chaotic signals

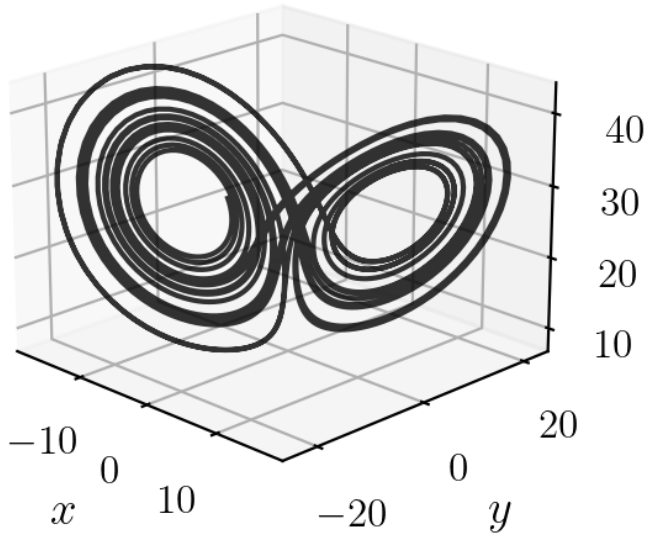
- Statistics of chaotic signals not affected by butterfly effect
  → Prediction of statistics becomes possible



Attractor: set of values that q takes asymptotically

Nonlinear autonomous dynamical system:

$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$
$$\boldsymbol{\phi}(\boldsymbol{x}, 0) = \boldsymbol{\phi_0}$$



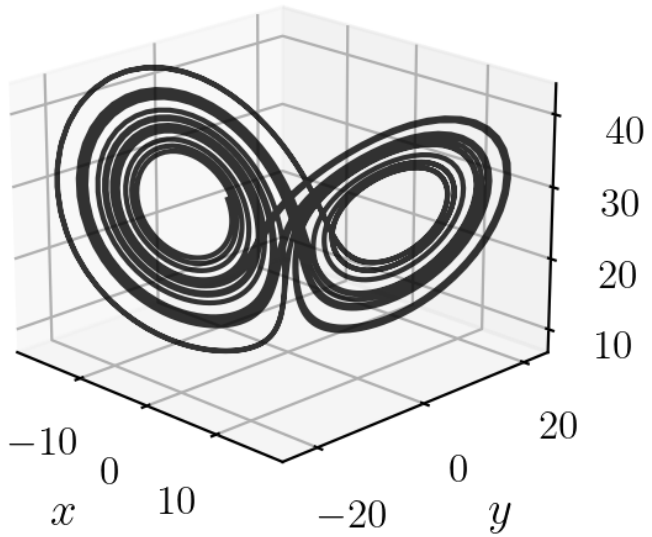
Attractor: once you are on the attractor, you remain on it

"Strange" attractor:

- Zero-measure in its embedding space
- Fractal dimension

Some indicator of chaos

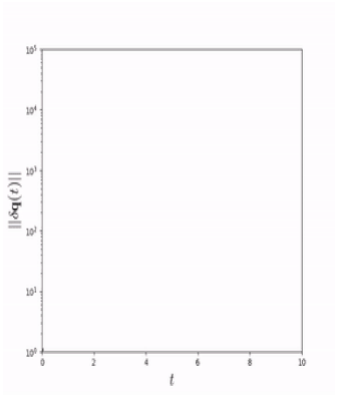
Characteristics of attractor

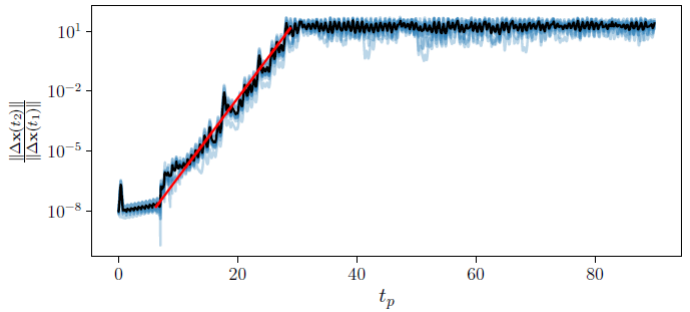


Estimate of the dimension of the attractor:

- Kaplan-Yorke dimension (can be estimated using the Lyapunov spectrum, not elaborated here)

Largest Lyapunov exponent





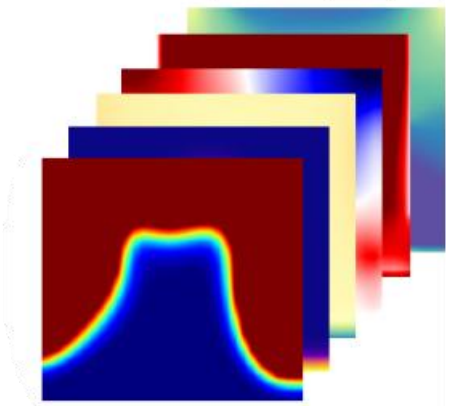
- λ_1 Lyapunov exponent: “Rate of divergence” of those two trajectories
- Lyapunov time: $t_p = 1/\lambda_1$

→ Indicator of “predictability” of the chaotic system

Typical dynamical modelling in turbulent (reacting) flows: Flow state prediction

- Flow state prediction:

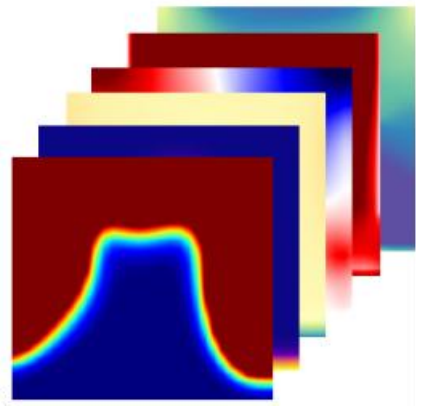
$\boldsymbol{\phi}(0)$



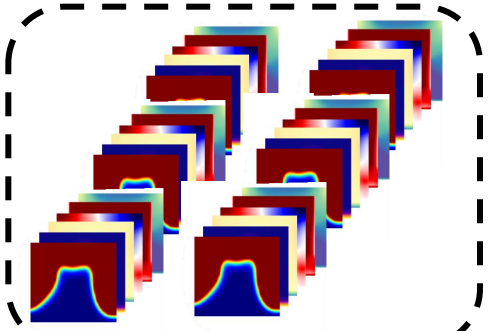
$\boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$

Typically done with CFD

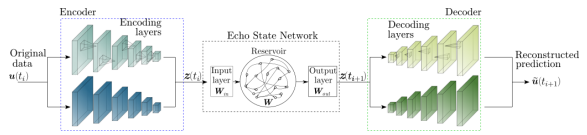
$\boldsymbol{\phi}(n)$



$\{\boldsymbol{\phi}(0), \ldots, \boldsymbol{\phi}(n)\}$

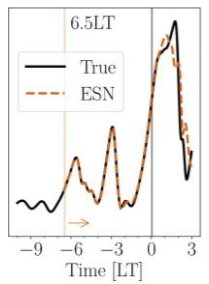


Development of a (faster) data-driven reduced-order model



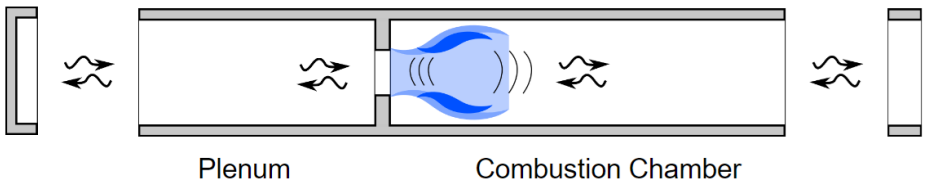
Applications:

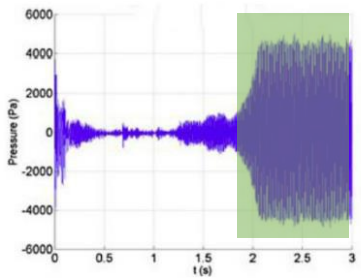
- Time-forecasting (for instability forecasting)
- Influence of forcing/geometry
- Model-predictive control



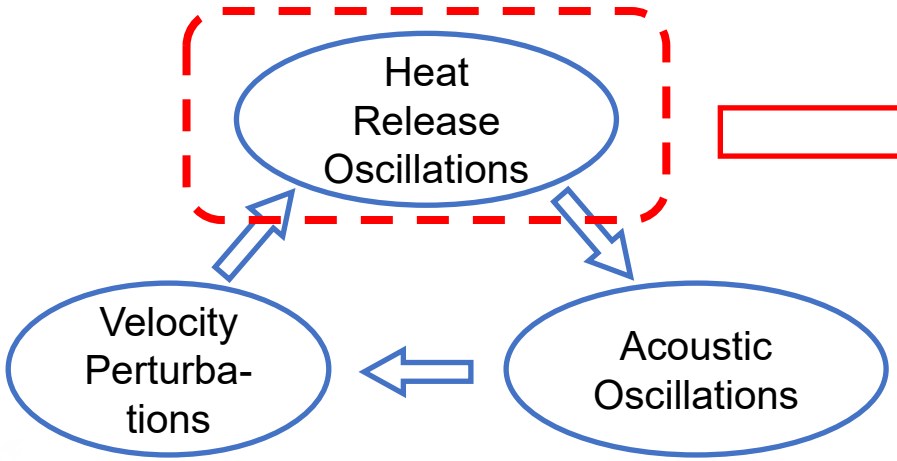
Typical dynamical modelling in turbulent (reacting) flows: Flame dynamics modelling

- Thermoacoustic instabilities: High amplitude pressure oscillations in premixed combustors

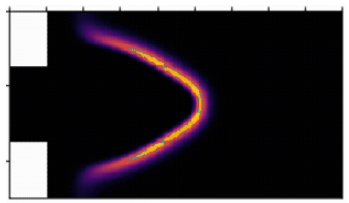




Thermoacoustic instabilities: result from coupling between HRR and acoustic fluctuations



$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$



$$\rightarrow \dot{\boldsymbol{Q}} = \mathcal{N}(\boldsymbol{\phi}, \boldsymbol{F})$$

$$\rightarrow \dot{\boldsymbol{Q}} = \mathcal{F}(\boldsymbol{F_u})$$

Modelling aim:

- Prediction of flow (or QoI) response to specific forcing

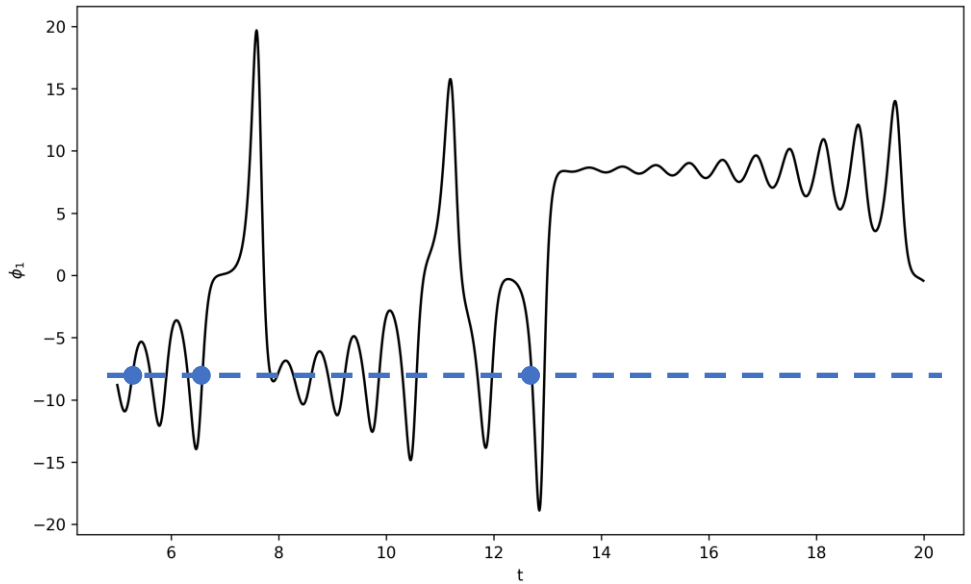
Outline

- Key Concepts in Dynamical Systems and Chaos
- **Machine Learning for Dynamical Systems**
  - **Sequential data and dynamical systems**
  - System Identification for LTI systems
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$
$$\boldsymbol{\phi}(\boldsymbol{x}, 0) = \boldsymbol{\phi_0}$$

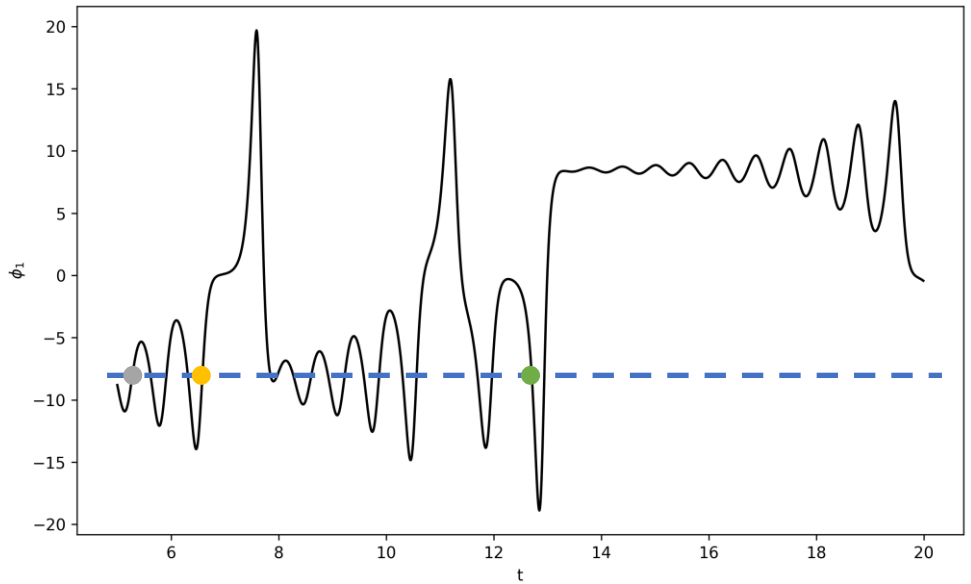


- System exhibit multiple time the same values

Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$
$$\boldsymbol{\phi}(\boldsymbol{x}, 0) = \boldsymbol{\phi_0}$$

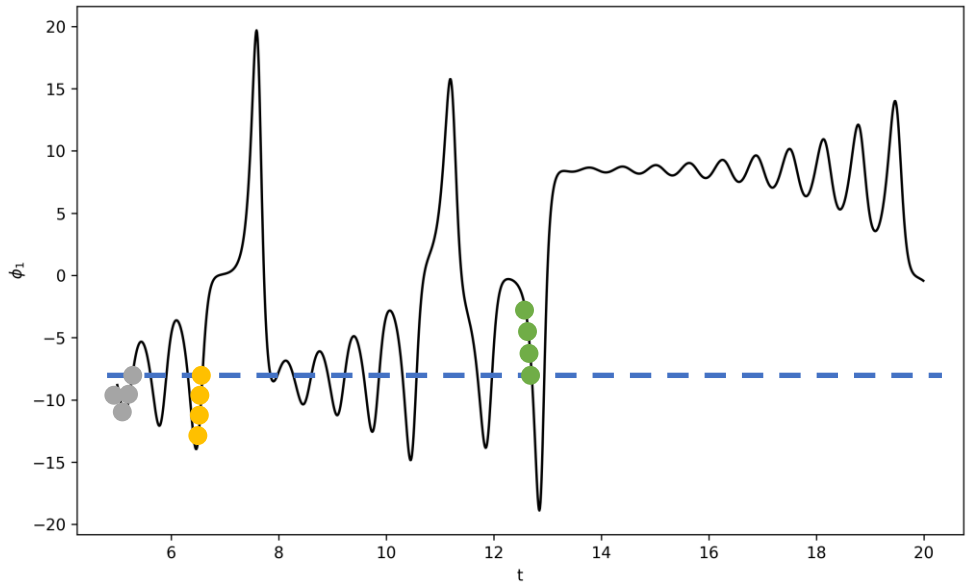


- System exhibit multiple time the same values
- But these are different data points

Dynamical systems produce sequential data

Nonlinear autonomous dynamical system:

$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$
$$\boldsymbol{\phi}(\boldsymbol{x}, 0) = \boldsymbol{\phi_0}$$

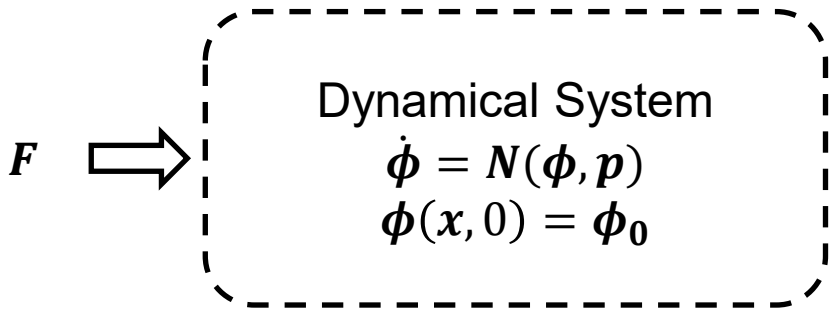


- System exhibit multiple time the same values
- But these are different data points
- The previous data provides the information

→ The *sequence* of data point is important

Slight change in notation

- Forcing considered as an “input” to the system



- Discrete-time notation:
  - $\boldsymbol{\phi}(n)$ or $\boldsymbol{\phi}_n$: state sampled at time t_n
- Objective of data-driven dynamical system modelling:
  - Based on some data of the dynamical system…
  - …Obtain a model that can forecast $\widetilde{\boldsymbol{\phi}}(n)$ (or a quantity of interest $Q(\widetilde{\boldsymbol{\phi}})$) which approximates the true evolution $\boldsymbol{\phi}(n)$ (or of $Q(\boldsymbol{\phi})$)

Data-driven Modelling of Dynamical Systems

- Has strong ties to “system identification”
- Has a strong history (before “machine learning”)
- Finds its root in signal processing and control theory
  - Generally developed to model systems from measured data
  - Mostly applied to linear, time-invariant systems
  - See e.g. Rabiner and Gold (1975); Bellanger (2000); Ljung (1999) for overview
- Extension to nonlinear dynamical systems
  - With recent neural networks-based approaches

[Theory and Application of Digital Signal Processing, Rabiner & Gold, Prentice Hall 1975.]
[Digital Processing of Signals, Bellanger, Wiley 2000.]
[System Identification – Theory for the User, Ljung, Prentice Hall 2000.]

Outline

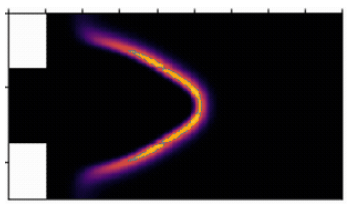
- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - **System Identification for LTI systems**
  - Data-driven nonlinear dynamical system modelling
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

Brief introduction to system identification

- Presentation based on review by Polifke
- Focus is on “system’s response to a signal”

Generally: $\boldsymbol{s}$ ⇨ System/Plant ⇨ $\boldsymbol{r}$

In our dynamical system notation: $\boldsymbol{F}$ ⇨ $\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p})$ ⇨ $\boldsymbol{r} = Q(\boldsymbol{\phi})$

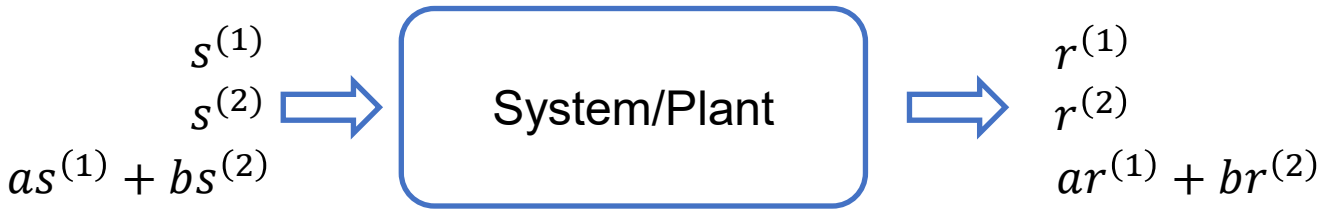
In the thermoacoustic context: $\boldsymbol{u'}$ ⇨  ⇨ $\dot{Q}$

[Black-box system identification for reduced order model construction, W. Polifke, Ann. Nucl. Energy (67), 2014.]

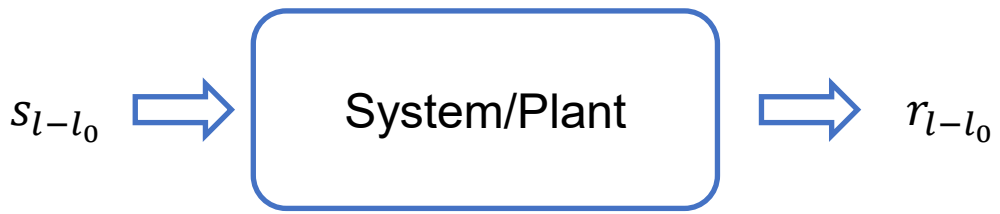
Brief introduction to system identification

- Assumption: causal linear time-invariant (LTI) system (and for single input/output)

Linearity



Time invariance



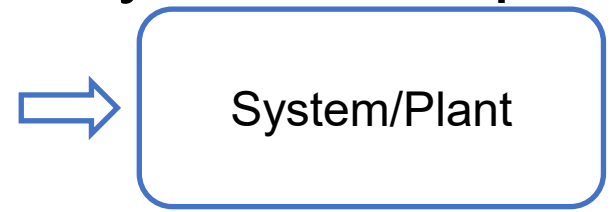
Causal: response r_l depends only on the present/past signal, i.e.:

r_l does not depend on s_m for $m > l$

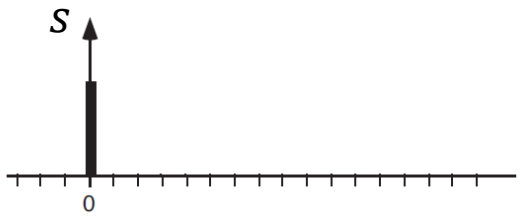
Brief introduction to system identification

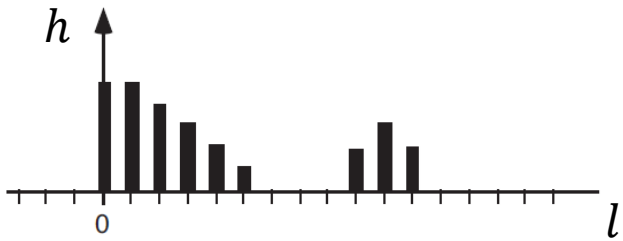
- Impulse response: system's response to "impulse"

$$s(l) \equiv i_l = \begin{cases} 1, l = 0 \\ 0, l \neq 0 \end{cases}$$



$r \equiv h_l$: impulse response





- Impulse response: characterizes the LTI system's response to *any* signal

$$r_l = \sum_{k=-\infty}^{\infty} h_k s_{l-k} \rightarrow \boldsymbol{r} = \boldsymbol{h} * \boldsymbol{s} \qquad \text{(convolution)}$$

For causal system: $h_k = 0$ for $k < 0$ → Fully characterizes the system's dynamics

$$r_l = \sum_{k=0}^{\infty} h_k s_{l-k}$$

Brief introduction to system identification

- How can we estimate the impulse response h_k?
- First, consider the finite response ($k = 0, \dots, N$):

$$r_l \approx \sum_{k=0}^{N} h_k s_{l-k}$$

- Assume we have collected some signal/response of length L:

$$\boldsymbol{s} \equiv \{s_l\} \text{ and } \boldsymbol{r} \equiv \{r_l\} \; (l = 0, \dots, L)$$

- Correlation analysis can be used to estimate h_k

Brief introduction to system identification

- Consider the auto-correlation matrix $\boldsymbol{\Gamma}$ of $\boldsymbol{s}$ (estimated using L time lags)

$$\Gamma_{ij} \approx \frac{1}{L-N+1}\sum_{k=N}^{L} s_{k-i}s_{k-j}\,, \qquad i,j = 0,\dots,N$$

- Cross correlation vector $\boldsymbol{c}$

$$c_i \approx \frac{1}{L-N+1}\sum_{k=N}^{L} s_{k-i}r_k\,, \qquad i = 0,\dots,N$$

- One can show the following relation:

$$\boldsymbol{\Gamma h} = \boldsymbol{c} \text{ (Wiener-Hopf equation)}$$

$$\rightarrow \boldsymbol{h} = \boldsymbol{\Gamma}^{-1}\boldsymbol{c}$$

Brief introduction to system identification

$$\boldsymbol{h} = \boldsymbol{\Gamma}^{-1}\boldsymbol{c}$$

- Direct inversion of $\boldsymbol{\Gamma}$ can result in "over-fitting" of the impulse response coefficients (or may be ill-posed if $\boldsymbol{\Gamma}$ non-invertible)

- Regularization generally used - $\boldsymbol{h}$ such that:

$$\min(\|\boldsymbol{\Gamma h} - \boldsymbol{c}\|^2 + \alpha^2\|\boldsymbol{R h}\|^2)$$

$\boldsymbol{R}$: Regularization matrix (can be the identity matrix – Ridge regression)

Exact solution:

$$\rightarrow \boldsymbol{h}_\alpha = (\boldsymbol{\Gamma}^T\boldsymbol{\Gamma} + \alpha^2\boldsymbol{R}\boldsymbol{R}^T)^{-1}\boldsymbol{\Gamma}^T\boldsymbol{c}$$

Brief introduction to system identification

$$\rightarrow \boldsymbol{h} = \boldsymbol{\Gamma}^{-1}\boldsymbol{c} \text{ or } \boldsymbol{h}_\alpha = (\boldsymbol{\Gamma}^T\boldsymbol{\Gamma} + \alpha^2\boldsymbol{R}\boldsymbol{R}^T)^{-1}\boldsymbol{\Gamma}^T\boldsymbol{c}$$

- If s is white noise, $\boldsymbol{\Gamma} = \mathbf{I}$ (identity matrix)

- Summary

(I) Some signal, e.g. white noise (or other signal with invertible auto-correlation) $\boldsymbol{s}$ ⇨ System/Plant (can be your flame) ⇨ $\boldsymbol{r}$ Record the response

(II) Compute $\boldsymbol{\Gamma}, \mathbf{c}$

(III) Deduce $\boldsymbol{h} = \boldsymbol{\Gamma}^{-1}\boldsymbol{c}$ or $\boldsymbol{h}_\alpha = (\boldsymbol{\Gamma}^T\boldsymbol{\Gamma} + \alpha^2\boldsymbol{R}\boldsymbol{R}^T)^{-1}\boldsymbol{\Gamma}^T\boldsymbol{c}$ $\boldsymbol{s}$ ⇨ $\boldsymbol{h}$ ⇨ $\boldsymbol{r}$

(IV) With $\boldsymbol{h}$, estimate $\boldsymbol{r}$ for any $\boldsymbol{s}$

Brief introduction to system identification

- Approach can be extended to multi-input/multi-output systems
- In the specific case of thermo-acoustic, interest is in the frequency response:

If $s_l(\omega) = e^{1_i \omega \Delta t l}$:

$$r_l = \sum_{k=-\infty}^{\infty} h_k e^{1_i \omega \Delta t (l-k)} = e^{1_i \omega \Delta t l} \sum_{k=-\infty}^{\infty} h_k e^{-1_i \omega \Delta t k}$$

$$\rightarrow F(\omega) \equiv \sum_{k=-\infty}^{\infty} h_k e^{-1_i \omega \Delta t k}$$

(Laplace/z-transform of $\boldsymbol{h}$, estimated at $z = e^{1_i \omega \Delta t}$)

$$\rightarrow r_l(\omega) = s_l(\omega) F(\omega)$$

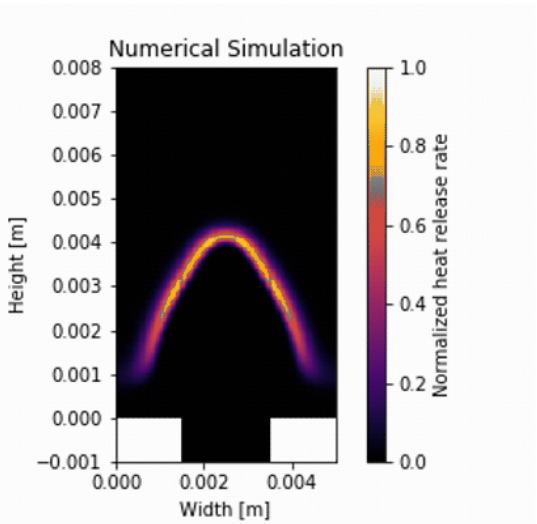
Brief introduction to system identification

- Practical considerations:
  - Turbulent fluctuations can mask “noise” the response signal, so masking/filtering of turbulent fluctuation may be necessary
  - “Optimal” excitation signal $\boldsymbol{s}$ (for acoustic identification):
    - Excitation should be uncorrelated with itself
    - Signal should be broadband, low-pass filtered with constant amplitude in frequency range of interest
- …

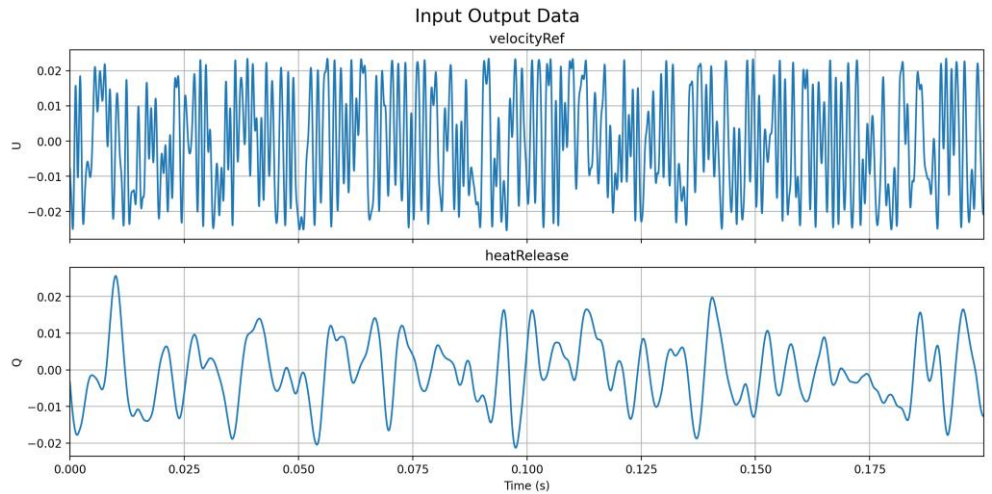
[Black-box system identification for reduced order model construction, W. Polifke, Ann. Nucl. Energy (67), 2014.]

Hands-on session – System Identification of linear Flame dynamics of a laminar slit flame

- https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026/blob/main/01_SystemIdentification.ipynb
- Premixed methane-air laminar slit burner (equivalence ratio of 0.8)
- Fully resolved with DNS



- Time series of input (velocity fluctuation) and output (heat release rate fluctuation) provided



- Application of System Identification to obtain flame frequency response

Kornilov et al. PROCI (2007)

Outline

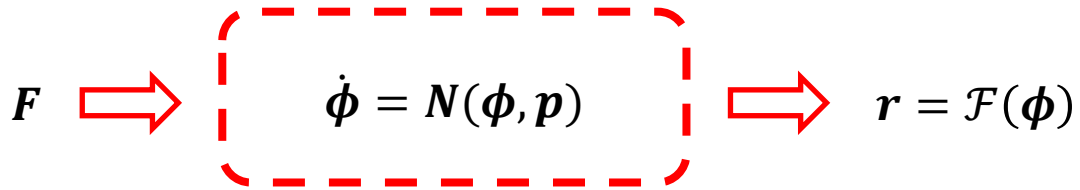
- Key Concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and dynamical systems
  - System Identification for LTI systems
  - **Data-driven nonlinear dynamical system modelling**
- Applications to reacting flows
  - Flame dynamics modelling
  - Reduced-order modelling

Outline for *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
3. Long Short-Term Memory Unit
4. Other RNNs/Advanced RNNs

Motivation

- So far: system identification for LTI system:



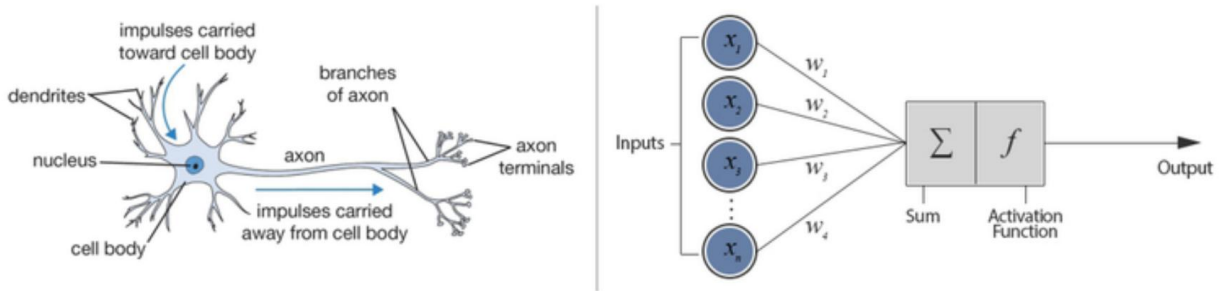
Strong assumptions remain:

→ Linear dynamical system

- What about when the system is nonlinear?
  - Various approaches possible (Volterra series, autoregressive models, …)
  - Focus here on: neural network-based nonlinear modelling

Neural network can be used as a basis for “any model”

- Neural networks
  - Inspired to reproduce the biological process of learning (late 1940s/early 1950s)



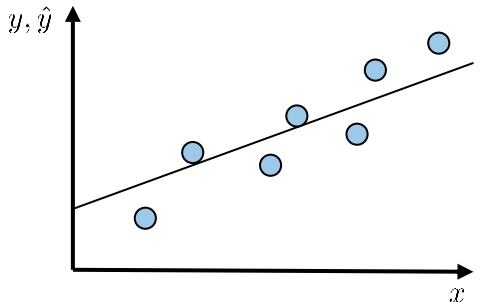
  - Universal approximator

    “Can approximate any kind of nonlinear function”
  - Very strong expressivity

    “Can represent a large variety of functions”

[Approximation capabilities of multilayer feedforward networks, Hornik, Neural Netw. (4), 1991.]
[Neural Network Learning: Theoretical Foundations, Anthony & Barnett, CUP 2009.]

The neuron introduces nonlinearity through its activation function

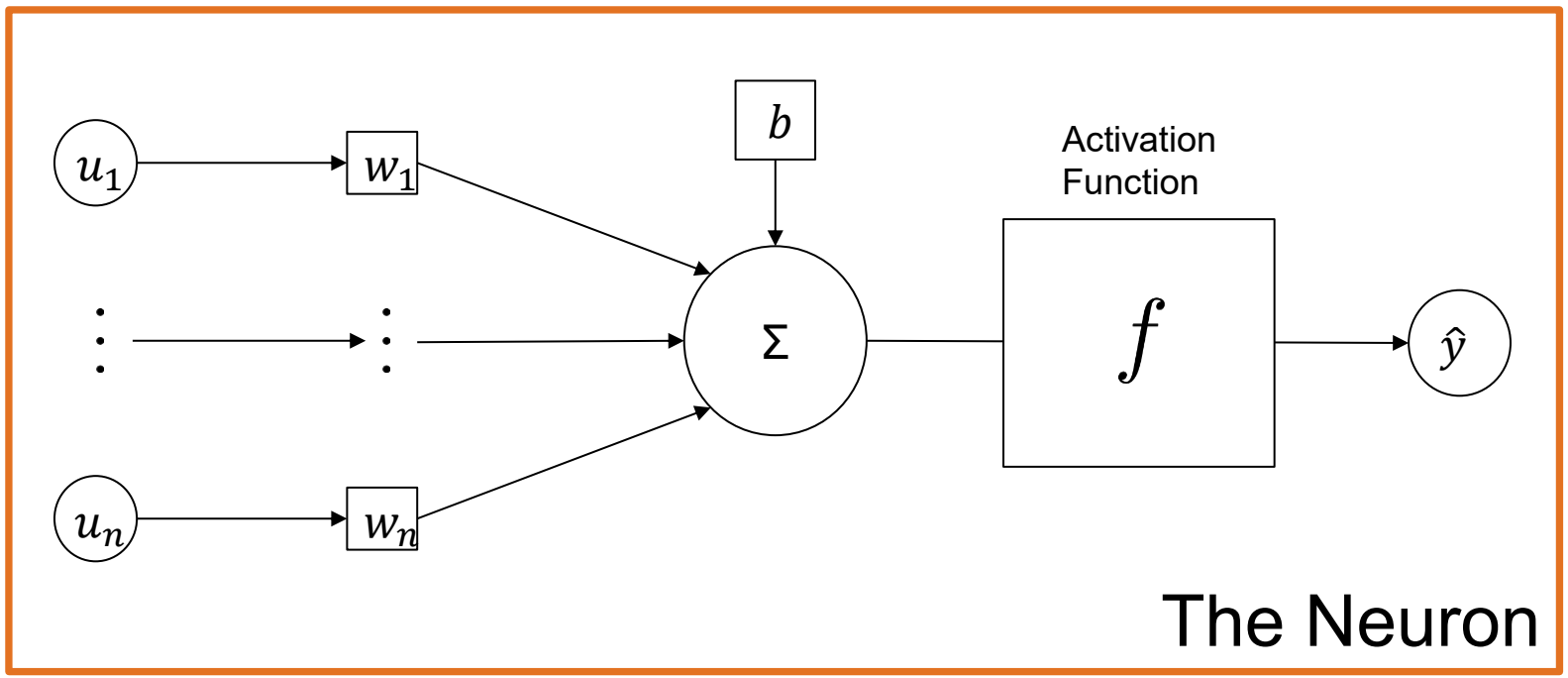


Standar linear regression:

$$\hat{y} = \sum_{i}^{n} w_i u_i + b_i = \boldsymbol{w} \cdot \boldsymbol{u} + \boldsymbol{b}$$

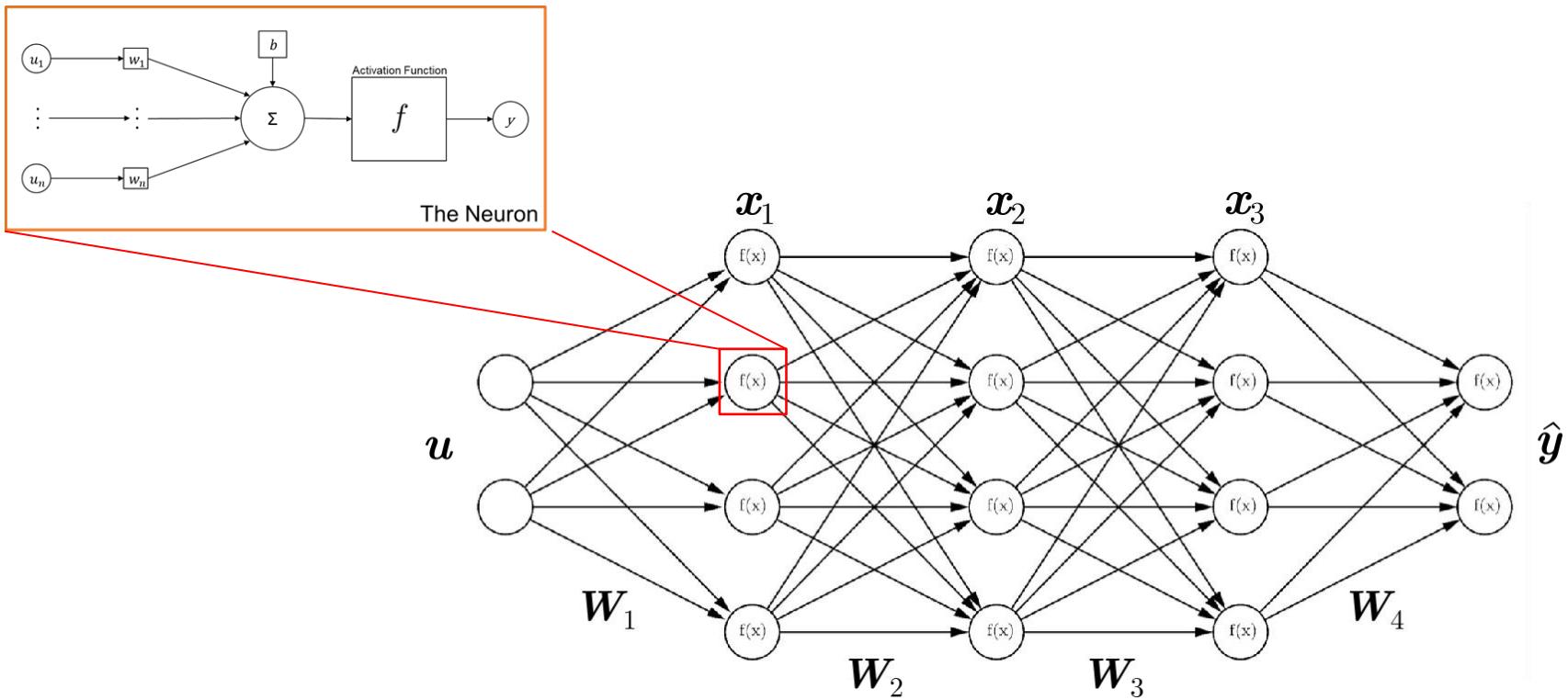
Nonlinearity introduced with activation function :

$$\hat{y} = f(\boldsymbol{w} \cdot \boldsymbol{u} + \boldsymbol{b})$$

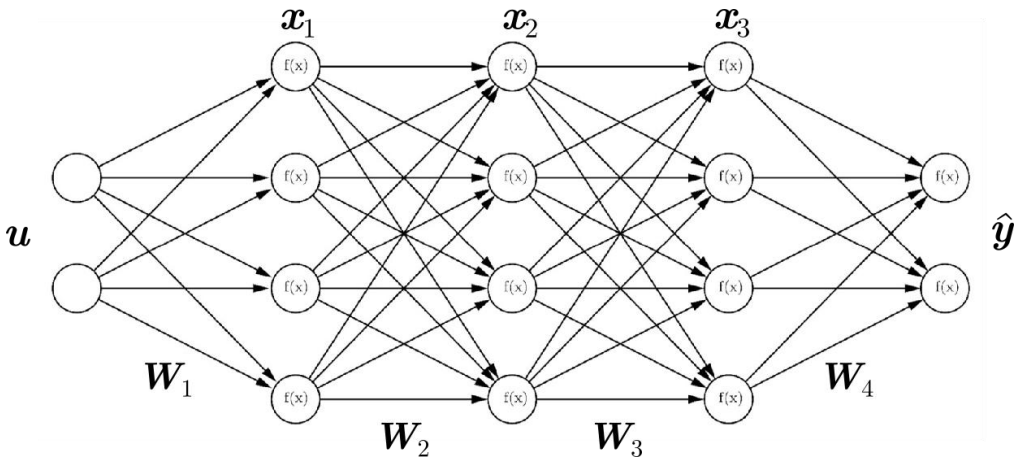


Feedforward neural network/Multilayer perceptron are obtained by chaining layers of neurons

- Dense deep neural network:
  - Fully connected neurons organised in layers
  - Enough neurons can approximate any function (universal approximator)



Learning/training is just optimizing the neurons' weights and biases



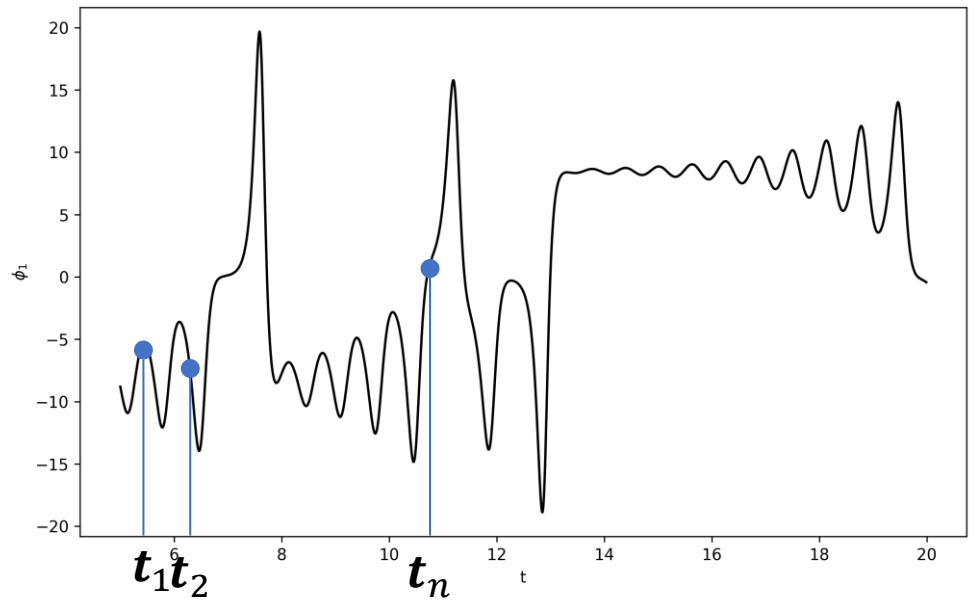
- Simplest type of task, data-regression: Minimization of the error:

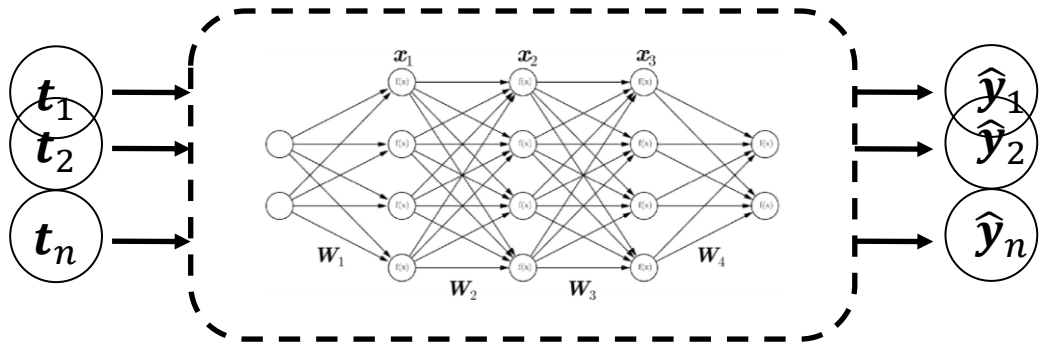
$$\min_{\boldsymbol{W},\boldsymbol{b}} \frac{1}{N} \sum_{i=1}^{N} \left|\left| \boldsymbol{y_i} - \hat{\boldsymbol{y}}_{\boldsymbol{i}} \right|\right|^2$$

→ Typically solved using gradient-based optimization where gradient obtained by automatic differentiation

Naive approach to sequence processing

- How to deal with sequence?
- Possible approach: time as input



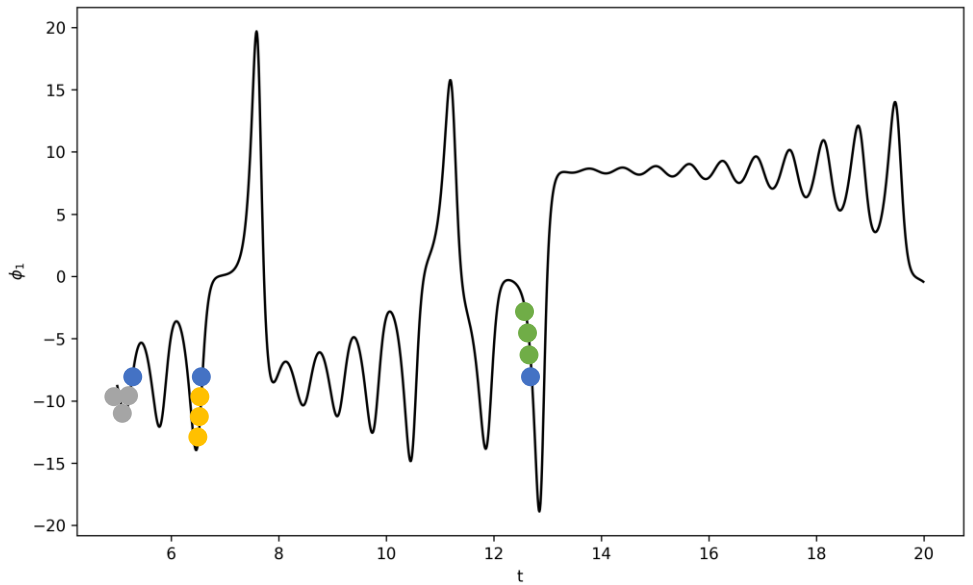


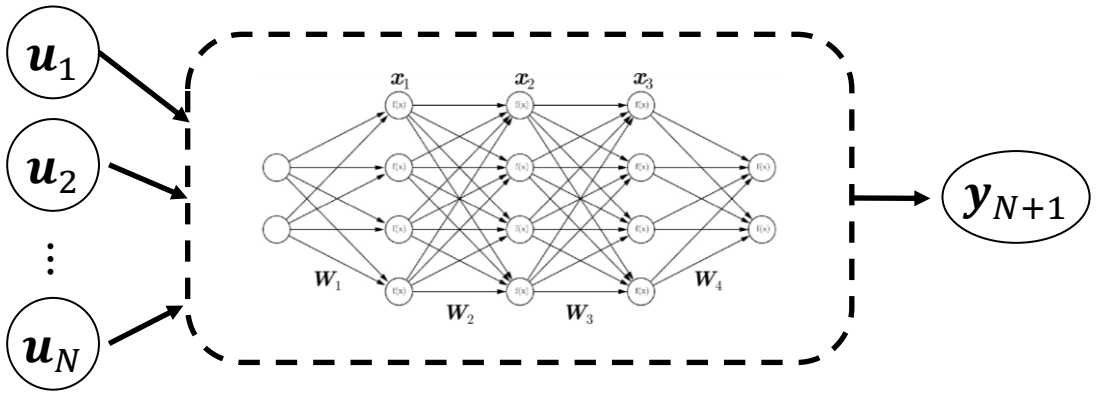
→ If input is time, no sense of "ordering"

→ Neural network: "value indexing", no possibility to extrapolate

Another naive approach to sequence processing

- Possible approach:



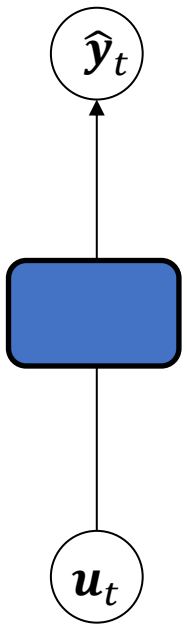


→ Use sequence of N past values as input

- Requires initial guess of time-dependency
- Can lead to large number of weights if long-term dependency required

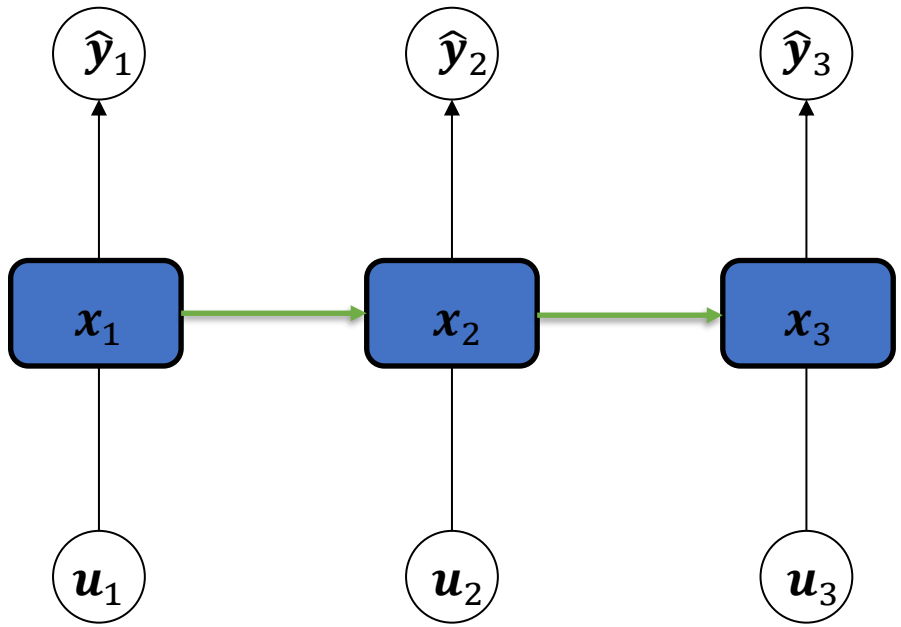
How to include time-dependency

Standard neuron

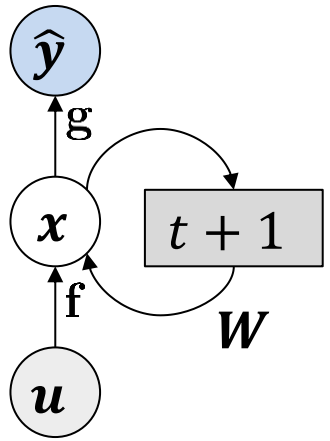


$$\hat{\boldsymbol{y}}_t = \mathcal{F}(\boldsymbol{u}_t)$$

Time-handling neuron



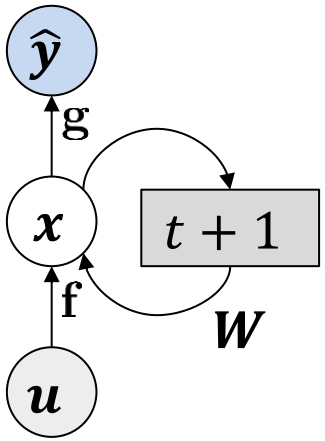
$$\hat{\boldsymbol{y}}_t = \mathcal{F}(\boldsymbol{u}_t, \boldsymbol{x}_{t-1})$$



$$\boldsymbol{x}_t = \boldsymbol{f}(\boldsymbol{x}_{t-1}, \boldsymbol{u}_t)$$
$$\widehat{\boldsymbol{y}_t} = \boldsymbol{g}(\boldsymbol{x}_t)$$

$\boldsymbol{x}$: internal/hidden state
$\boldsymbol{u}$: input
$\hat{\boldsymbol{y}}$: output

Recurrent neural network uses a “memory” to retain past information



$$x_t = f(x_{t-1}, u_t; W)$$
$$\widehat{y_t} = g(x_t; W_g)$$

x: internal/hidden state
u: input
$\hat{y}$: output

Recurrent neural network

- Have a “state/memory” which is retained and evolves
- Designed to deal with time series

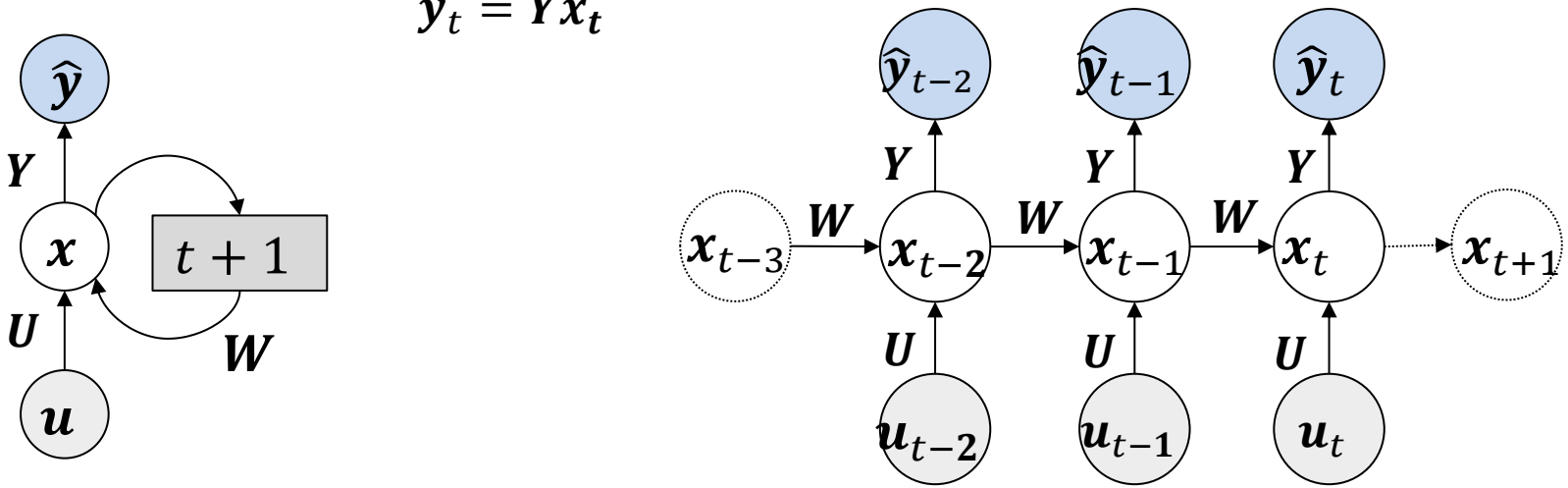
- Wrap up: Is a state space/dynamical system that can be time-advanced whose parameters have to be learned

Outline for *Data-driven nonlinear dynamical system modelling*

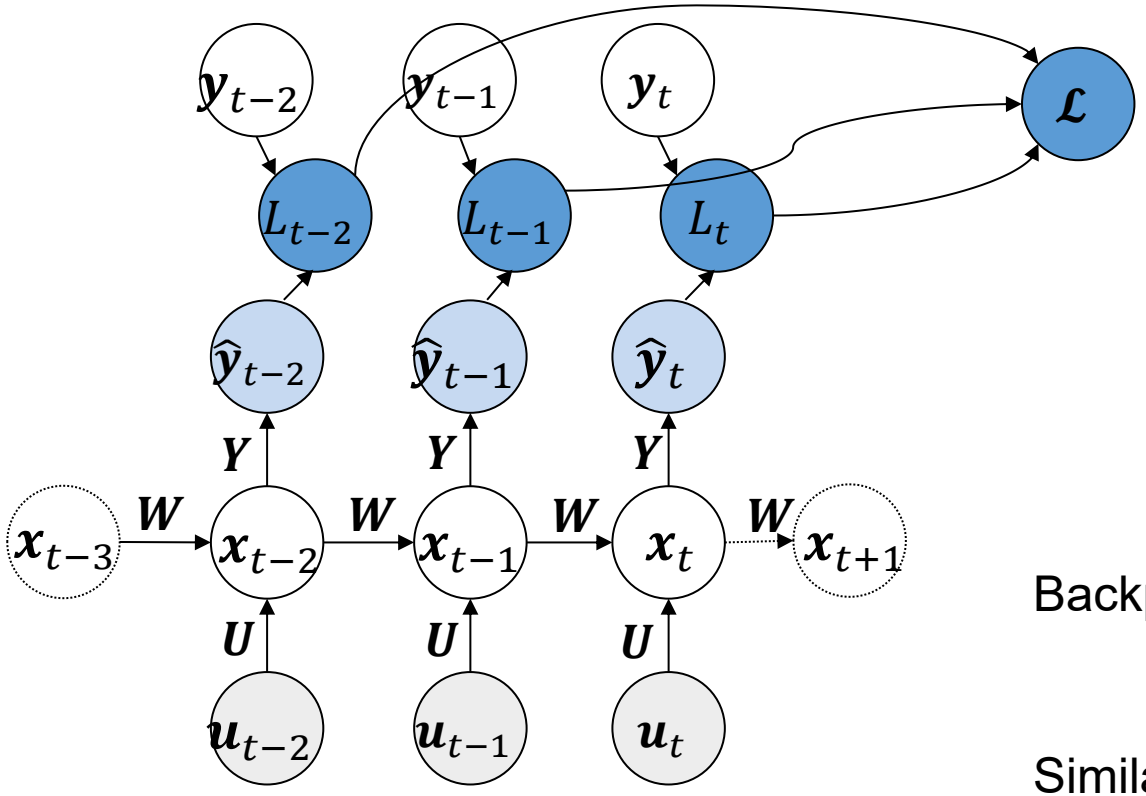
1. Motivation and General Concepts
2. **Backpropagation through Time**
3. Long Short-Term Memory Unit
4. Other RNNs/Advanced RNNs

Backpropagation in time: Unfolding the graph

$$\boldsymbol{x}_t = \tanh(\boldsymbol{W}\boldsymbol{x}_{t-1} + \boldsymbol{U}\boldsymbol{u}_t)$$
$$\widehat{\boldsymbol{y}_t} = \boldsymbol{Y}\boldsymbol{x}_t$$



Backpropagation in time: Unfolding the graph



Loss over multiple timesteps:

$$\mathcal{L} = \sum_{t-\tau}^{t} L_k$$

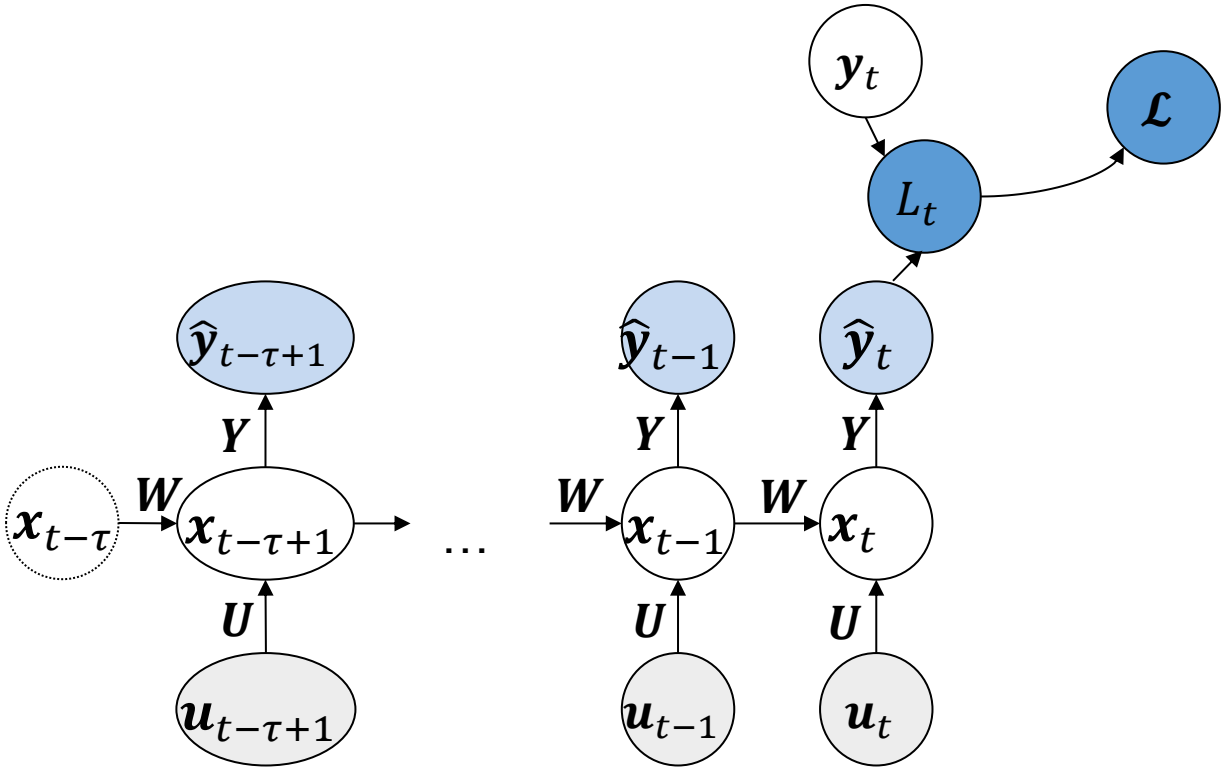
Backpropagation:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_t}\frac{\partial \boldsymbol{x}_t}{\partial \boldsymbol{W}} + \cdots + \frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_{t-\tau}}\frac{\partial \boldsymbol{x}_{t-\tau}}{\partial \boldsymbol{W}}$$

Similar to deep network:

#timesteps ≈ #layers

Long-term issues of backpropagation in time



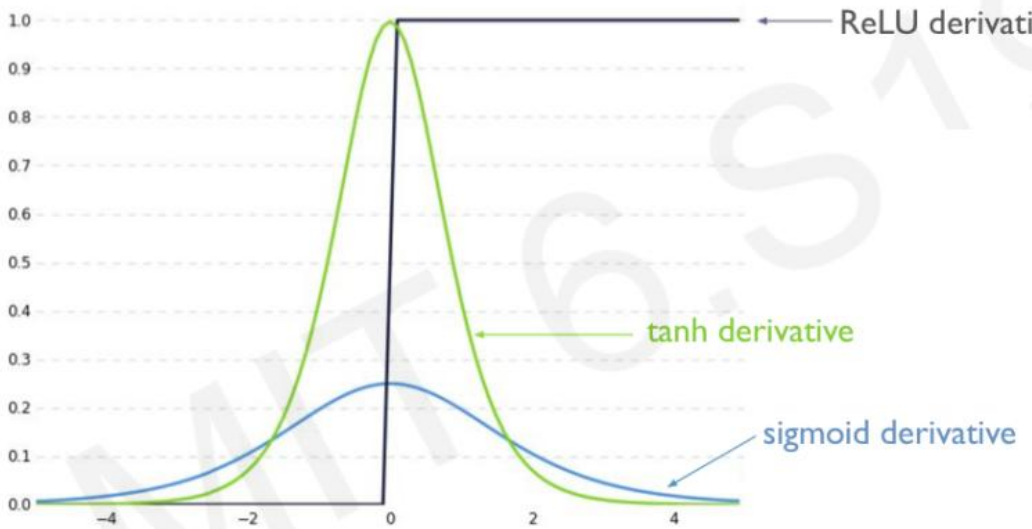
$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{x}_{t-\tau}} = \frac{\partial \sum L_t}{\partial \boldsymbol{x}_{t-\tau}} \rightarrow \frac{\partial L_t}{\partial \boldsymbol{x}_{t-\tau}} = \frac{\partial L_t}{\partial \boldsymbol{x}_t} \frac{\partial \boldsymbol{x}_t}{\partial \boldsymbol{x}_{t-\tau}}$$

Difficulty:
The gradient with respect to past goes through many applications of $\boldsymbol{W}$ and activation function if long-term dependency.

So, stability of gradient calculation depends on:
- Activation function
- Spectral radius of $\boldsymbol{W}$.
  $\boldsymbol{\rho}(\boldsymbol{W}) > 1$: exploding gradient
  $\boldsymbol{\rho}(\boldsymbol{W}) < 1$: vanishing gradient

How to overcome this long-term dependency issue?

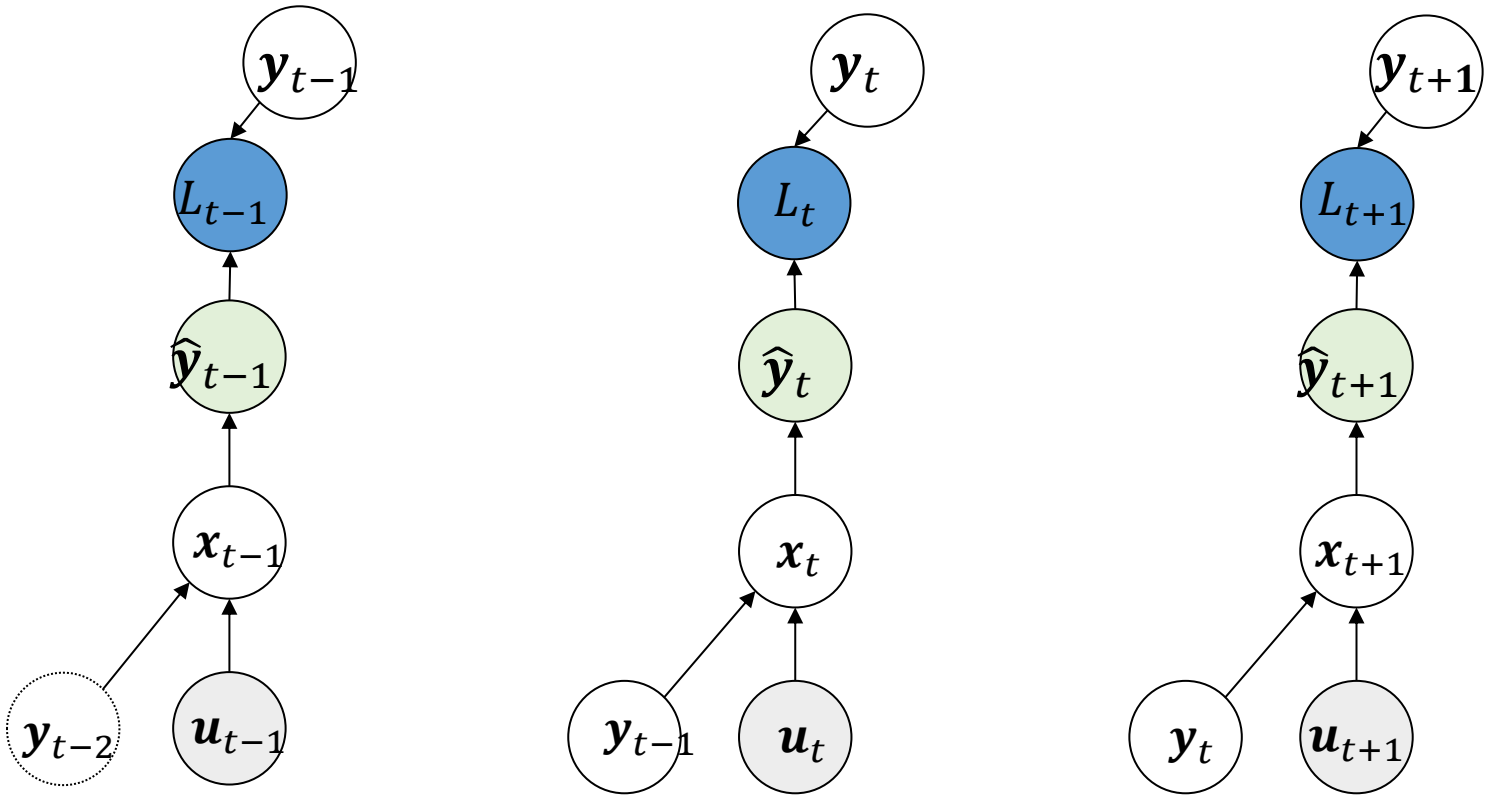
1. Use activation function that don't decrease f' too much (ReLU)



2. Initialize your Weight matrix $\boldsymbol{W}$ such that $\rho(\boldsymbol{W}) = 1$ (identity matrix)
3. Use more complex recurrent unit with gates to control the information flow (LSTM)

Teacher forcing can decouple the graph

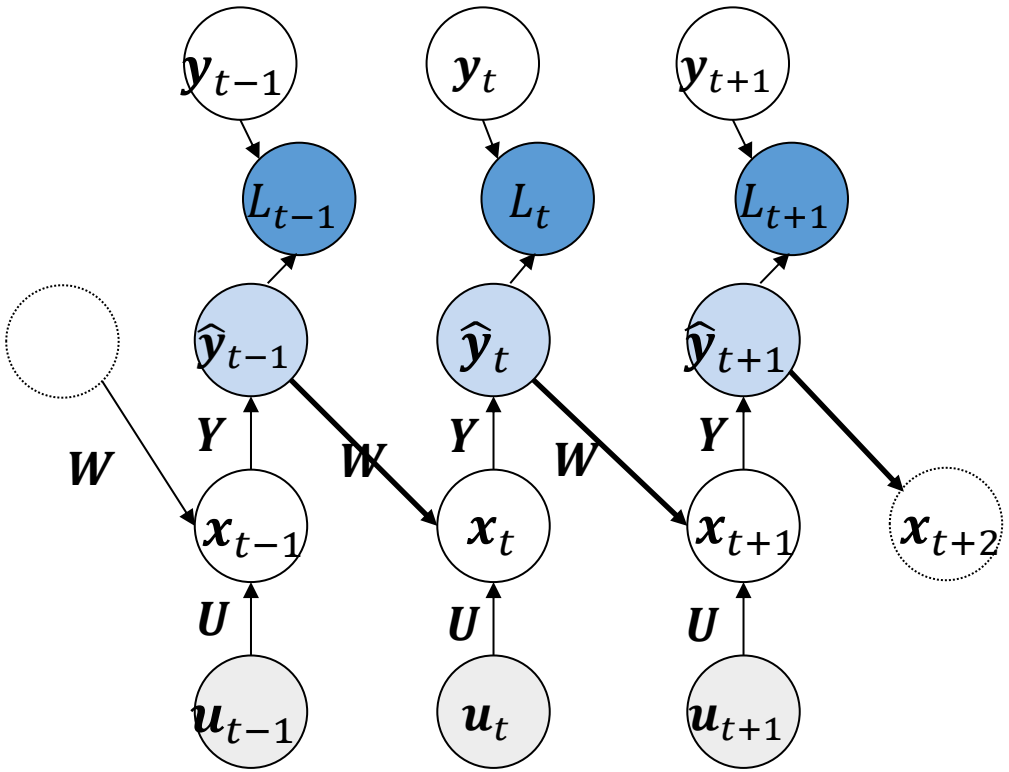
- To facilitate training we can feed the next target output as input and split the graph



! But does not train for long-term prediction !

Auto-regressive training: Feedback of the prediction back as input

Auto-regressive training:



More expensive training but more robust

Recurrent Neural Network – Wrap up

- RNN: Dynamical system with free parameters
  - Similarly to feedforward neural networks, RNN are universal approximators *of dynamical systems*
- Can be trained using Backpropagation Through Time (as usual)
  - But depth of backpropagation depends on number of time-step dependency
  - Teacher-forcing can be used to decouple the graph

[Recurrent Neural Networks Are Universal Approximators, Schäfer & Zimmermann, LNCS-CANN (4131), 2006]

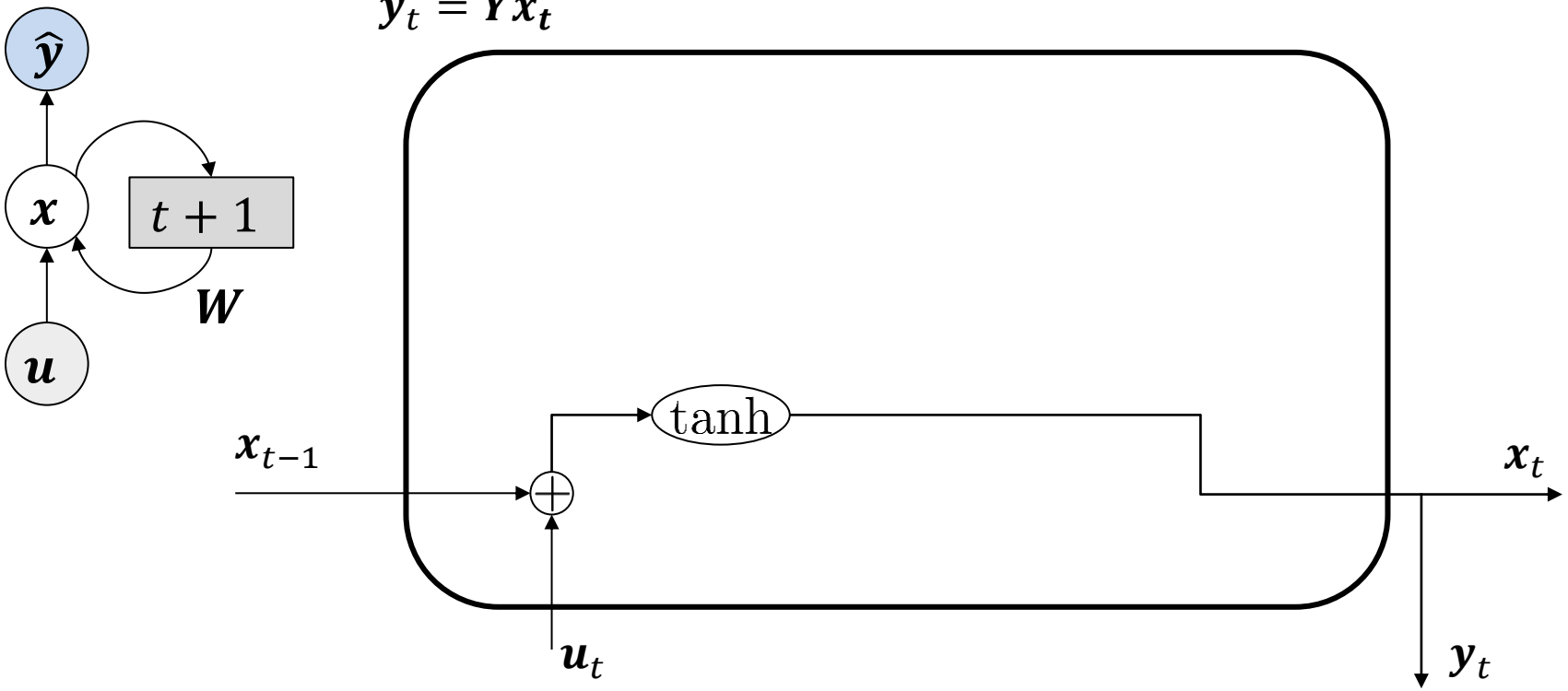
Outline for *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
3. **Long Short-Term Memory Unit**
4. Other RNNs/Advanced RNNs

Simple RNN

$$\boldsymbol{x}_t = \tanh(\boldsymbol{W}\boldsymbol{x}_{t-1} + \boldsymbol{U}\boldsymbol{u}_t)$$

$$\widehat{\boldsymbol{y}_t} = \boldsymbol{Y}\boldsymbol{x}_t$$



No long-term dependency!

Long-Short Term Memory

- Idea of not updating the whole hidden state each time
- Protect the state from being overwritten by useless information
- Be selective in
  - What to write (input gate) $\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i \boldsymbol{x}_{t-1} + \boldsymbol{U}_i \boldsymbol{x}_t + \boldsymbol{b}_i)$
  - What to read (output gate) $\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o \boldsymbol{x}_{t-1} + \boldsymbol{U}_o \boldsymbol{x}_t + \boldsymbol{b}_o)$
  - What to forget (forget gate) $\boldsymbol{f}_t = \sigma\left(\boldsymbol{W}_f \boldsymbol{x}_{t-1} + \boldsymbol{U}_f \boldsymbol{x}_t + \boldsymbol{b}_f\right)$

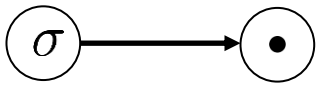
$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tanh(\boldsymbol{W}_x \boldsymbol{x}_{t-1} + \boldsymbol{U}_x \boldsymbol{u}_t + \boldsymbol{b}_x)$$

$$\boldsymbol{x}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t)$$

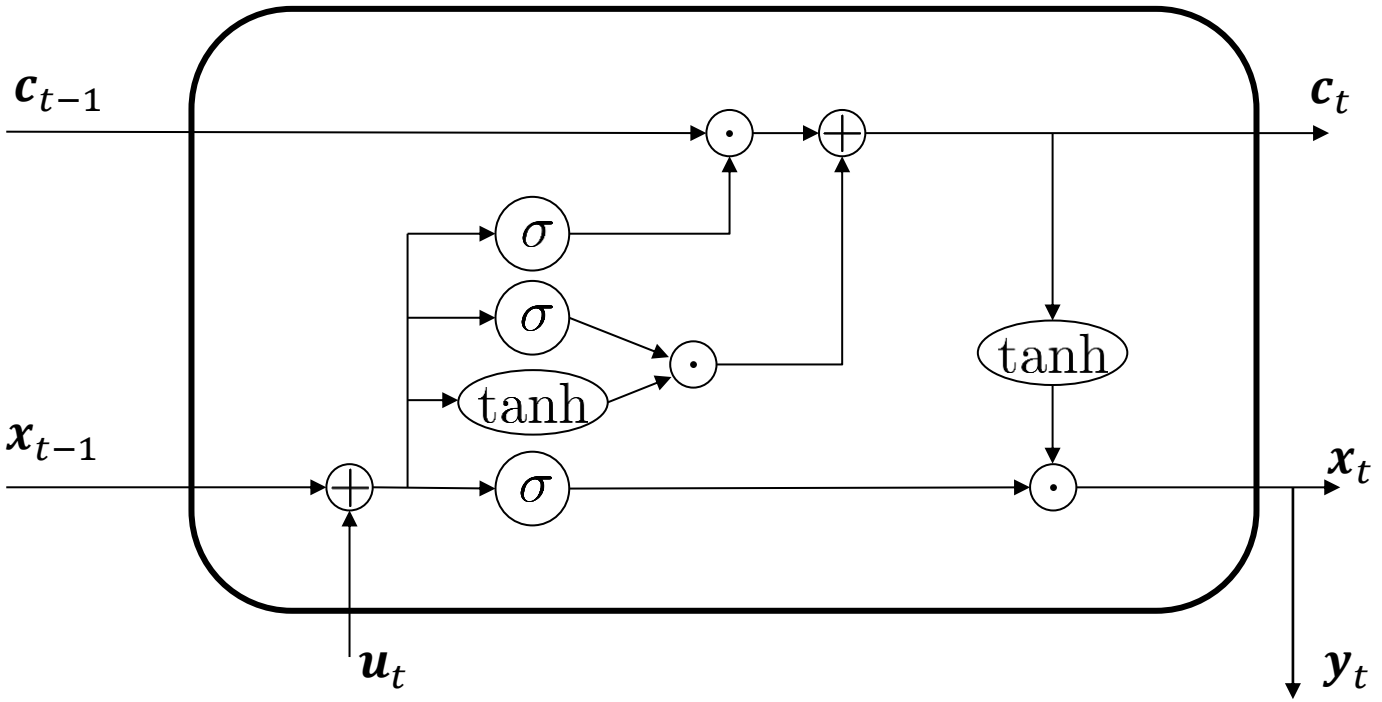
[Hochreichter & Schmidhuber, Neural Computation (1997)]

Long-Short Term Memory Unit

Gate: add/remove information with sigmoid

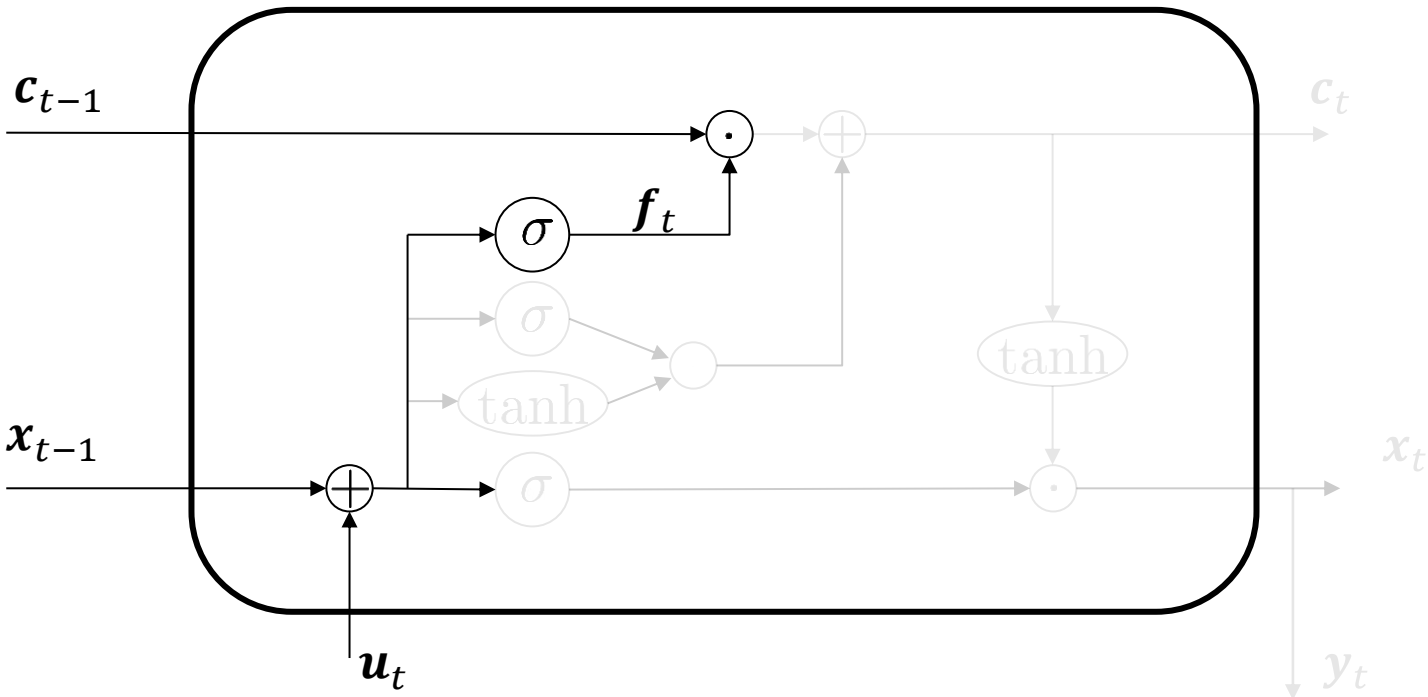


1. Forget
2. Store
3. Update
4. Output



LSTM - Forget

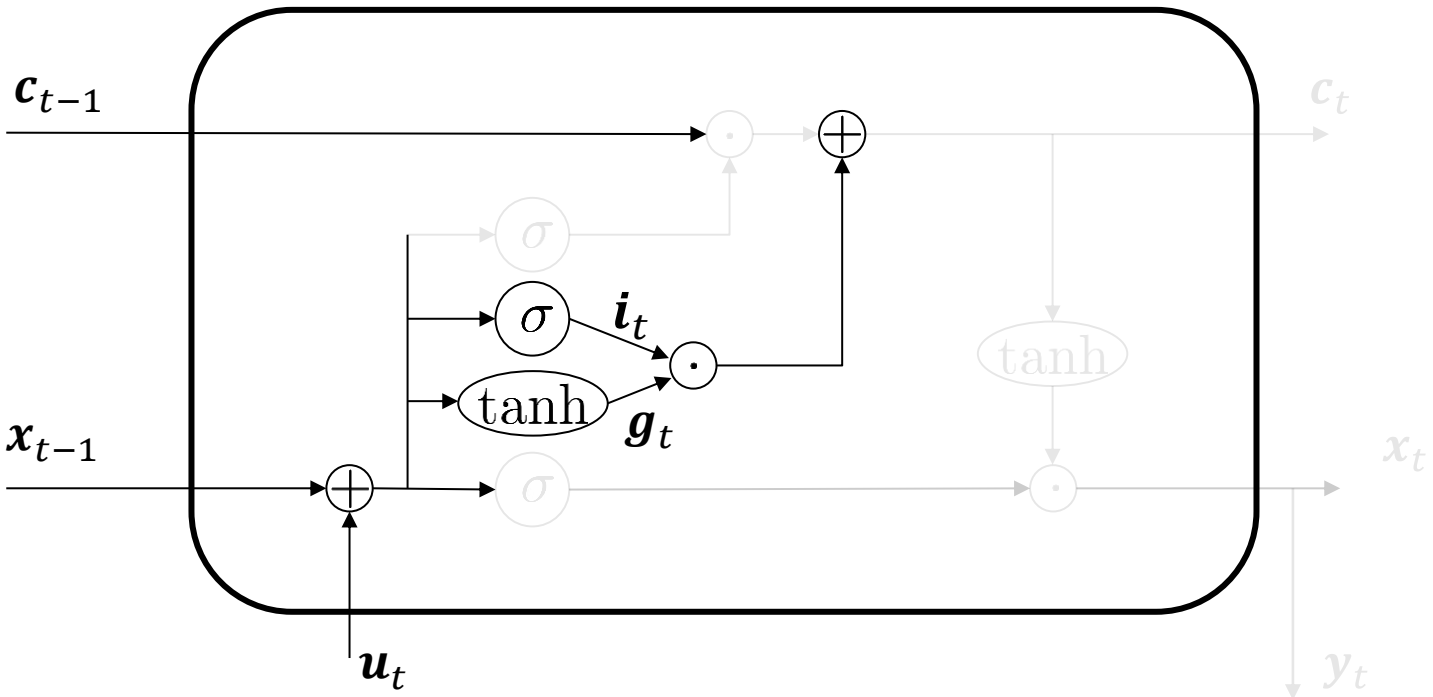
1. **Forget**
2. Store
3. Update
4. Output



$$f_t = \sigma\left(W_f x_{t-1} + U_f u_t + b_f\right)$$

LSTM – Input (store)

1. Forget
2. **Store**
3. Update
4. Output



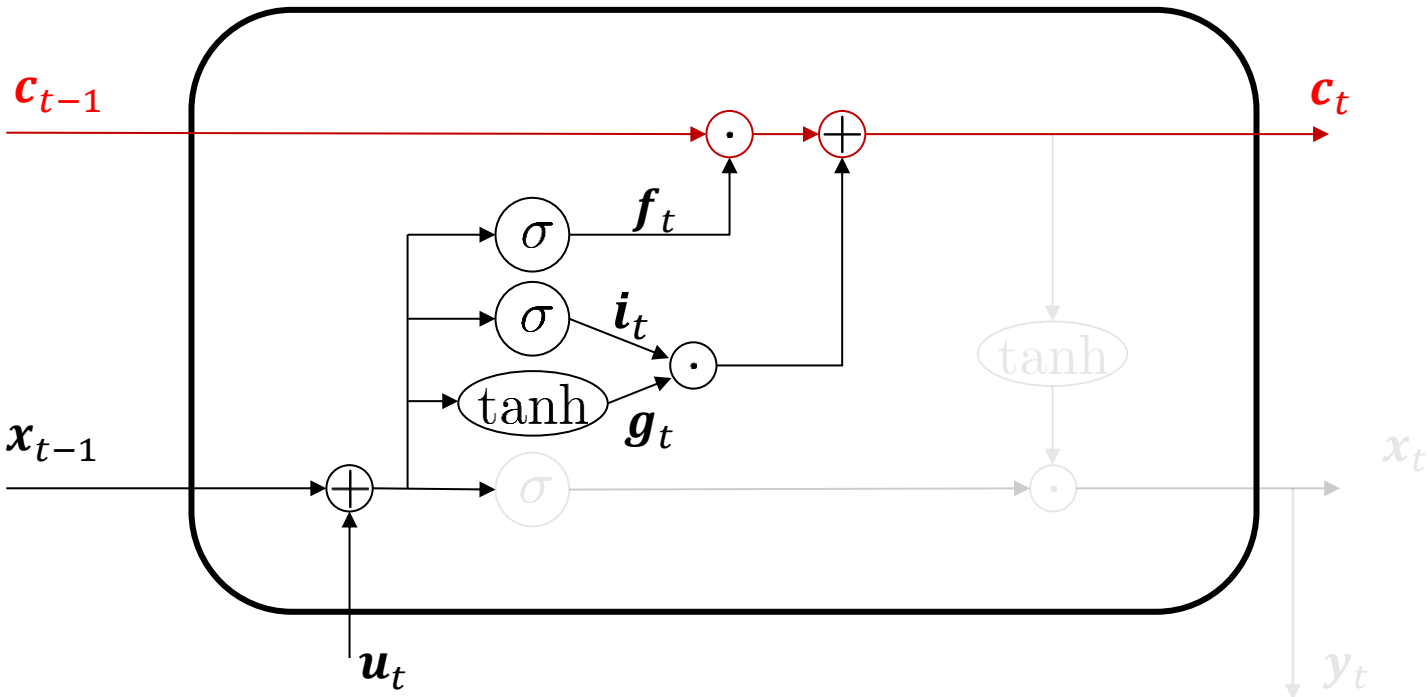
$$i_t = \sigma(W_i x_{t-1} + U_i u_t + b_i)$$

$$g_t = \tanh(W_x x_{t-1} + U_x u_t + b_x)$$

$$i_t \odot g_t$$

LSTM – Update

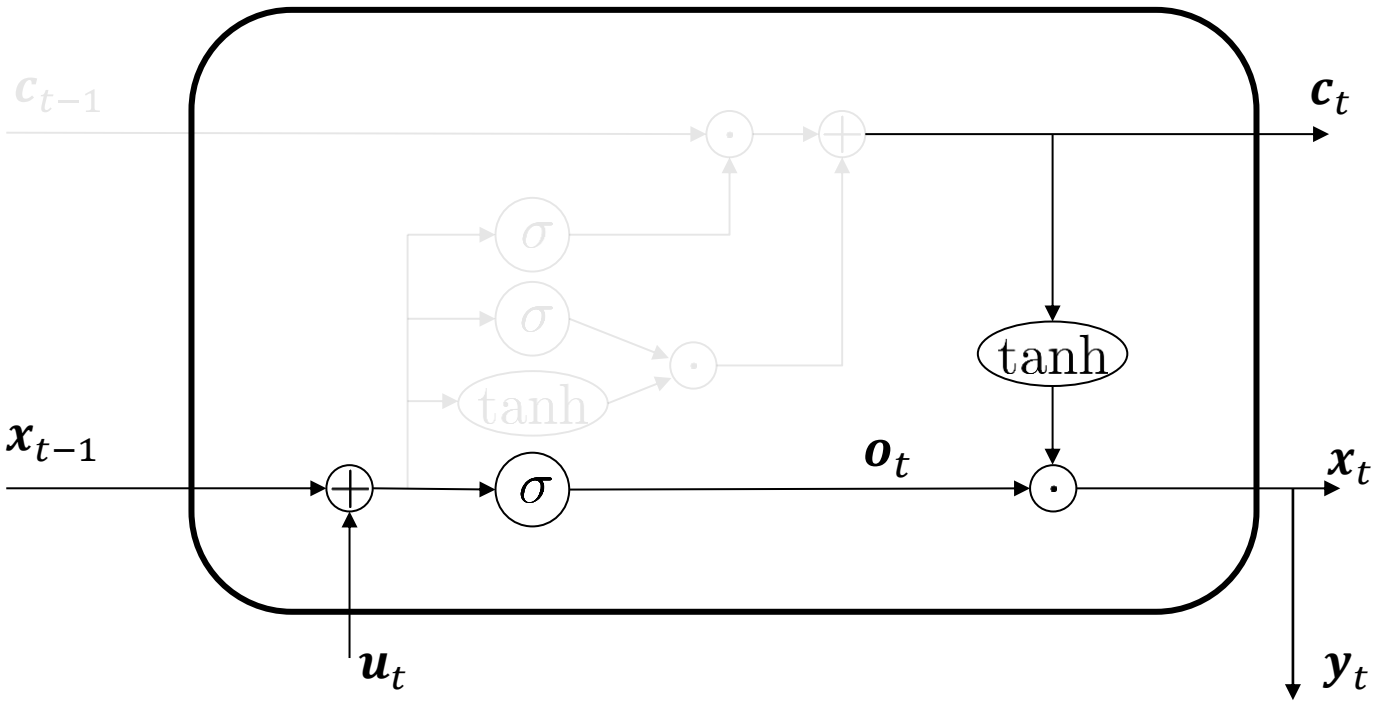
1. Forget
2. Store
3. **Update**
4. Output



$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

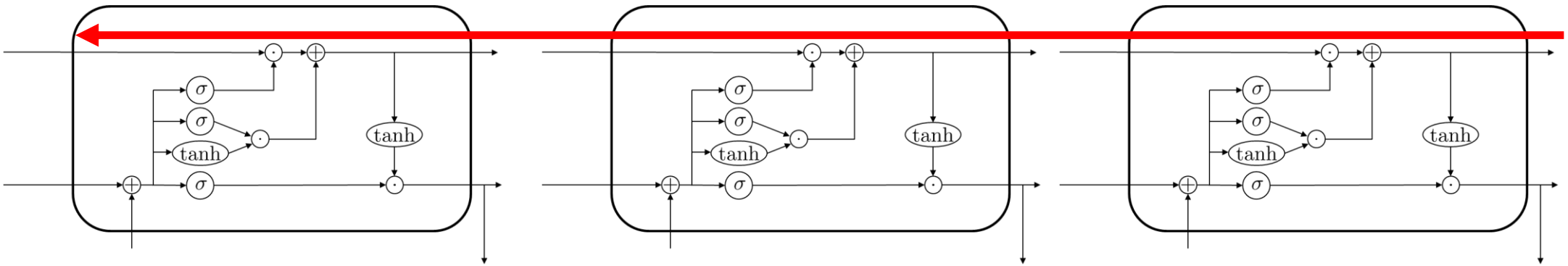
LSTM - Output

1. Forget
2. Store
3. Update
4. **Output**



$$o_t = \sigma(W_o x_{t-1} + U_o u_t + b_o)$$
$$x_t = o_t \odot \tanh(c_t)$$

LSTM allows for the uninterrupted gradient calculation



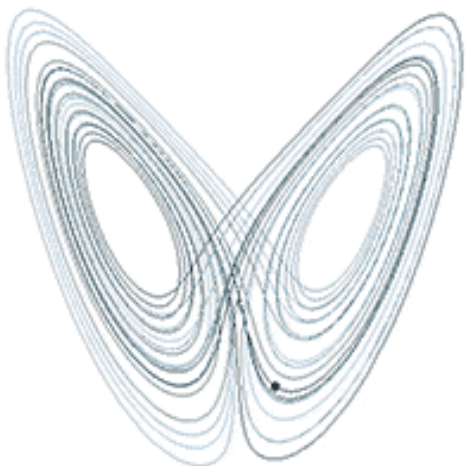
Hands-on session - Data-driven modelling of Lorenz System with MLP and LSTM

- https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026/blob/main/02_MLP_Lorenz.ipynb
- https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026/blob/main/03_LSTM_Lorenz.ipynb

- Lorenz system: developed originally as a model of atmospheric convection

$$\frac{dx}{dt} = \sigma(y - x), \frac{dy}{dt} = x(\rho - z) - y, \frac{dz}{dt} = xy - \beta z$$

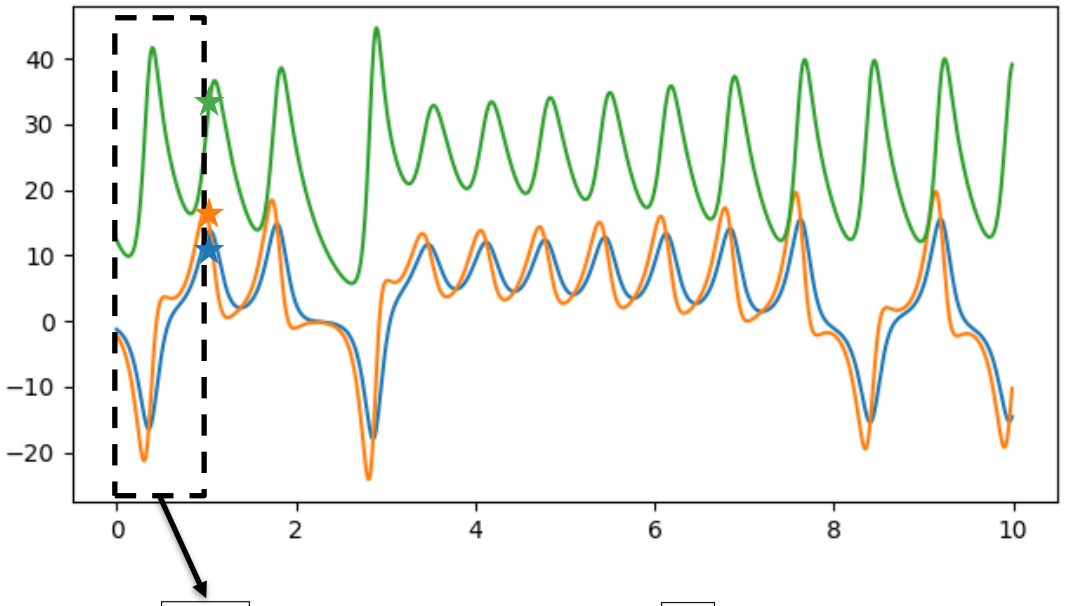
- Simplified model that exhibits chaotic behaviour.



$$\rho = 28, \sigma = 10, \beta = 8/3$$

Hands-on session - Data-driven modelling of Lorenz System with MLP and LSTM

Data: Time series of the Lorenz system



input

$\boldsymbol{x}_{t-N}$

$\vdots$

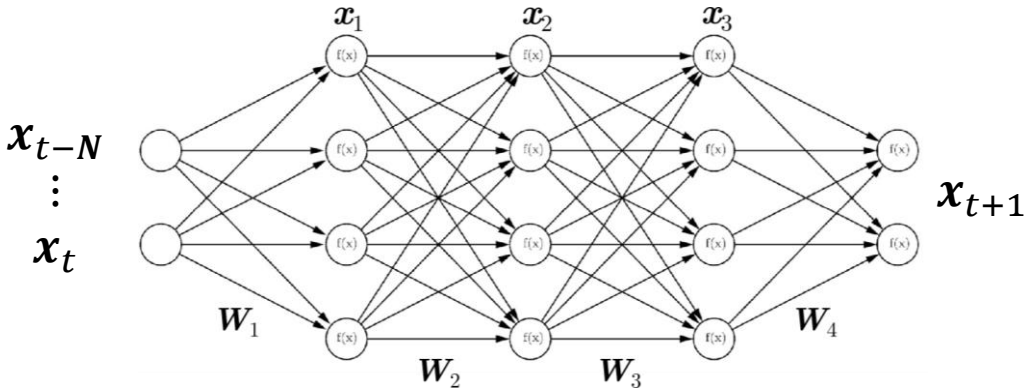
$\boldsymbol{x}_{t-1}$

target

$\boldsymbol{x}_t$

Objective: predict next time step based on previous ones:

$$\boldsymbol{x}(t+1) = \mathcal{F}\big(\boldsymbol{x}(t), \dots, \boldsymbol{x}(t-N)\big)$$



Steps:

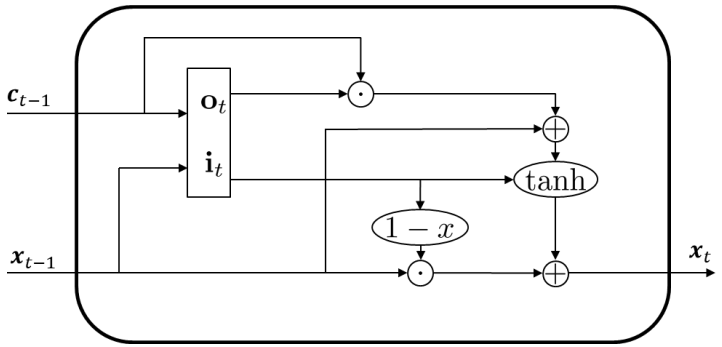
1. Restructure datasets into appropriate (input,pair)
2. Design the network
3. Train

Outline for *Data-driven nonlinear dynamical system modelling*

1. Motivation and General Concepts
2. Backpropagation through Time
3. Long Short-Term Memory Unit
4. **Other RNNs/Advanced RNNs**

Gated Recurrent Unit

- Alternative to LSTM: GRU (Gated Recurrent Unit)



Input: $\boldsymbol{i}_t = \sigma(\boldsymbol{W}_i \boldsymbol{x}_{t-1} + \boldsymbol{U}_i \boldsymbol{x}_t + \boldsymbol{b}_i)$

Forget: $\boldsymbol{f}_t = 1 - \boldsymbol{i}_t$

Output/read: $\boldsymbol{o}_t = \sigma(\boldsymbol{W}_o \boldsymbol{x}_{t-1} + \boldsymbol{U}_o \boldsymbol{x}_t + \boldsymbol{b}_o)$

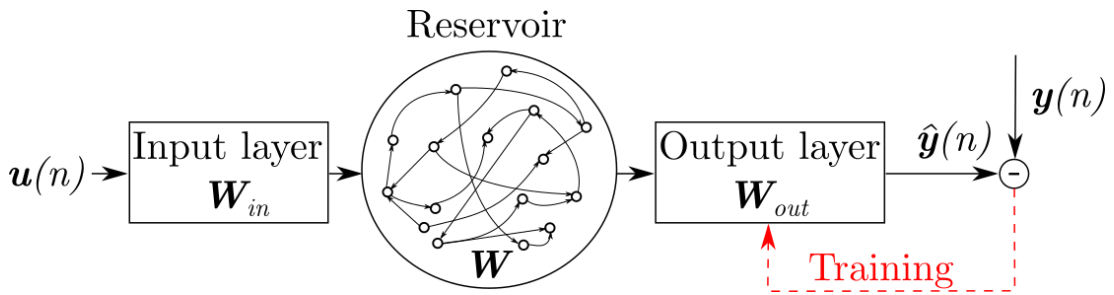
$$\boldsymbol{x}_{t+1} = \boldsymbol{f}_t \odot \boldsymbol{x}_{t-1} + \boldsymbol{i}_t \odot \tanh(\boldsymbol{W}_x \boldsymbol{x}_{t-1} + \boldsymbol{U}_x(\boldsymbol{o}_t \odot \boldsymbol{c}_{t-1}) + \boldsymbol{b}_x)$$

Similar to LSTM

[Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, Cho et al., EMNLP 2014.]

Reservoir computing is an alternative to LSTM

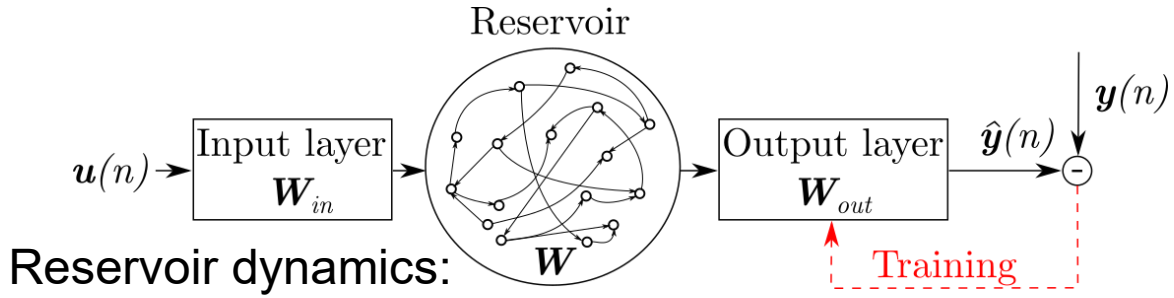
- Echo State Network:



- Static reservoir: high-dimensional fixed random system excited by the input
  - Base for “nonlinear expansion” of the system
  - Memory of the inputs
  - Mapping of the system’s dynamics on the reservoir states via “trained” linear readout
  - No need for backpropagation

[Echo state networks are universal, Grigoryeva & Ortega, Neural Netw. (108), 2018.]

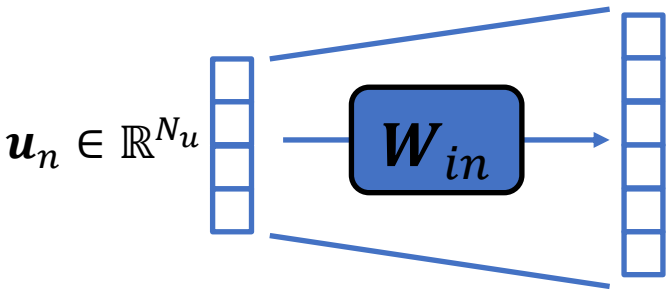
Reservoir computing is an alternative to LSTM



Reservoir dynamics:

$$x(n) = \tanh(W_{in}u(n) + Wx(n-1))$$

Project input to higher dimension

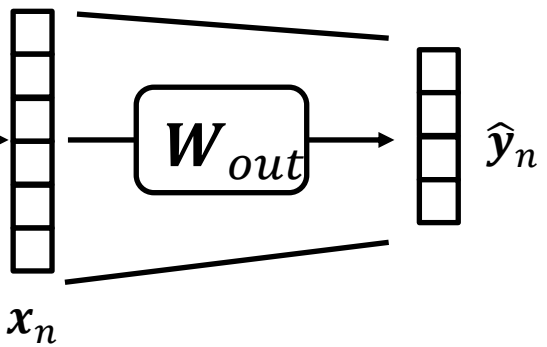


$W_{in}u_n$

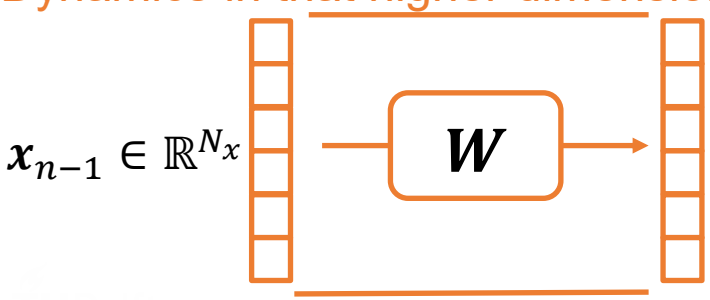
Combines input and states

tanh

Project back to target space



Dynamics in that higher dimension



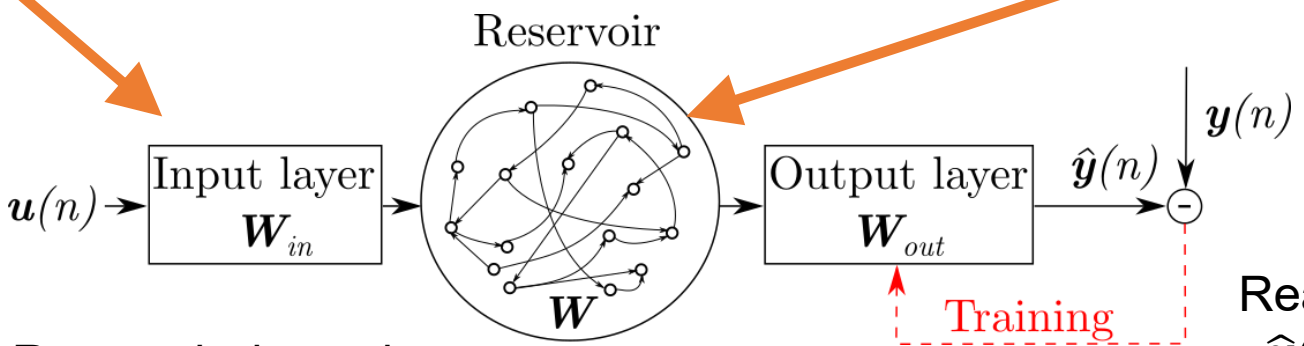
Wx_{n-1}

[Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication, Jaeger & Haas, Science (304), 2004.]

Echo State Network

$\boldsymbol{W}_{in}$: Sparse/Dense matrix randomly initialised from uniform distribution $[-\sigma_{in}, \sigma_{in}]$

$\boldsymbol{W}$:Sparse matrix randomly initialised from uniform distribution with set spectral radius $\Lambda \leq 1$ (echo state property)



Readout

$$\hat{\boldsymbol{y}}(n) = \boldsymbol{W}_{out}\boldsymbol{x}(n)$$

Reservoir dynamics:

$$\boldsymbol{x}(n) = \tanh(\boldsymbol{W}_{in}\boldsymbol{u}(n) + \boldsymbol{W}\boldsymbol{x}(n-1))$$

Training of $\boldsymbol{W}_{out}$ based on:

$$L = \frac{1}{N_y}\sum_{i=1}^{N_y}\frac{1}{N_t}\sum_{n=1}^{N_t}\left(\hat{y}_i(n) - y_i(n)\right)^2 + \gamma\frac{1}{N_y}\sum_{i=1}^{N_y}\left\|\boldsymbol{w}_{out,i}\right\|^2$$

Error on data

Regularity of $\boldsymbol{W}_{out}$

Training of the ESN is obtained by simple ridge regression

$$L = \frac{1}{N_y}\sum_{i=1}^{N_y}\frac{1}{N_t}\sum_{n=1}^{N_t}\left(\hat{y}_i(n) - y_i(n)\right)^2 + \gamma\frac{1}{N_y}\sum_{i=1}^{N_y}\left\|\boldsymbol{w}_{out,i}\right\|^2$$

- $\boldsymbol{W}_{out}$ obtained from ridge regression with

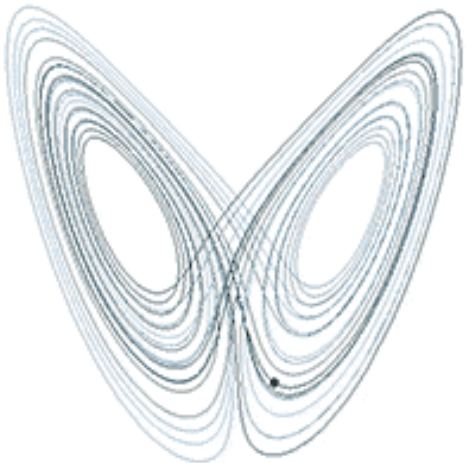
$$\boldsymbol{W}_{out} = \boldsymbol{Y}\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{X}^T + \gamma\boldsymbol{I})^{-1}$$

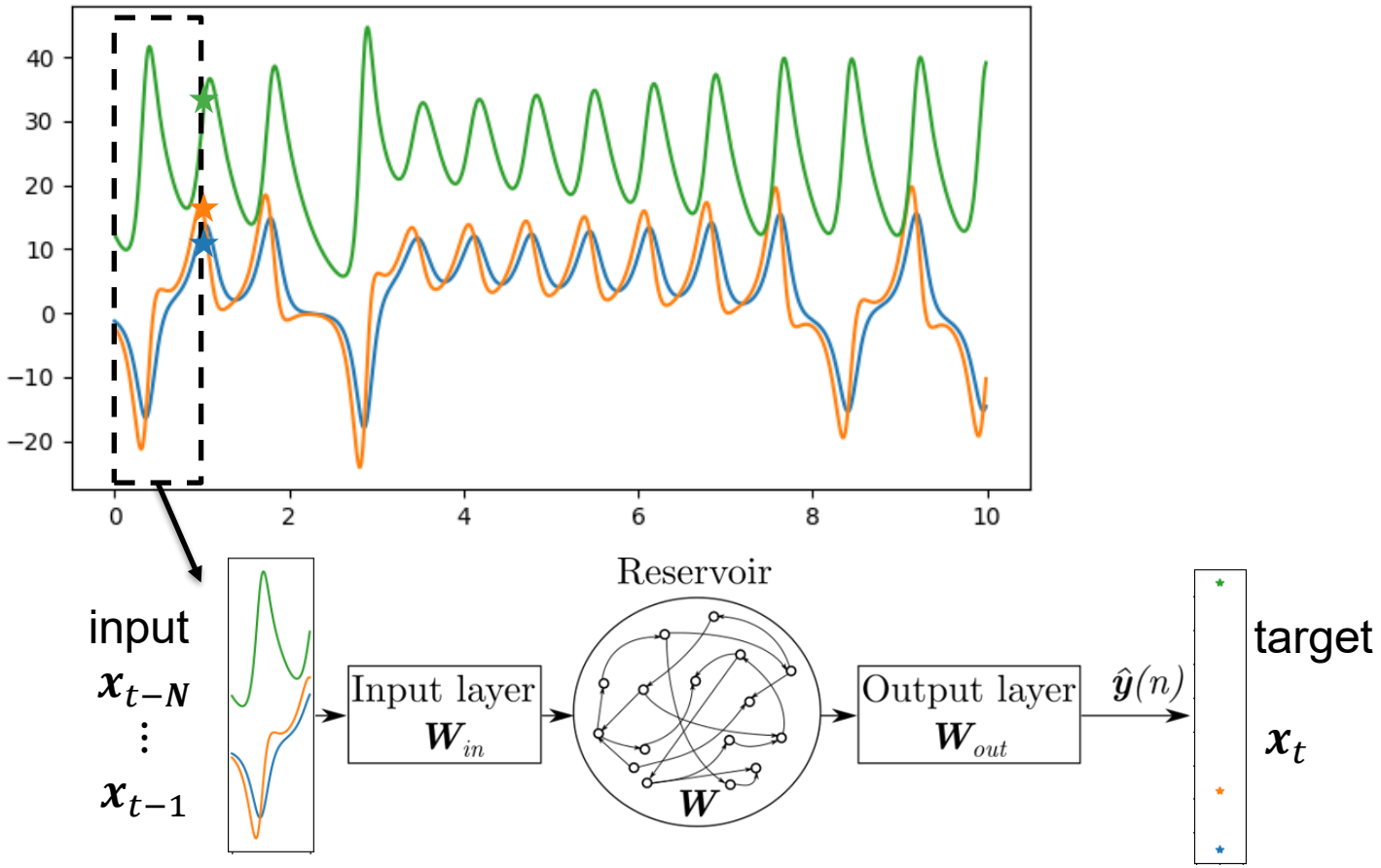
Where

- $\boldsymbol{X}$: Concatenated reservoir state
- $\boldsymbol{Y}$: Concatenated target data
- $\boldsymbol{I}$: Identity matrix

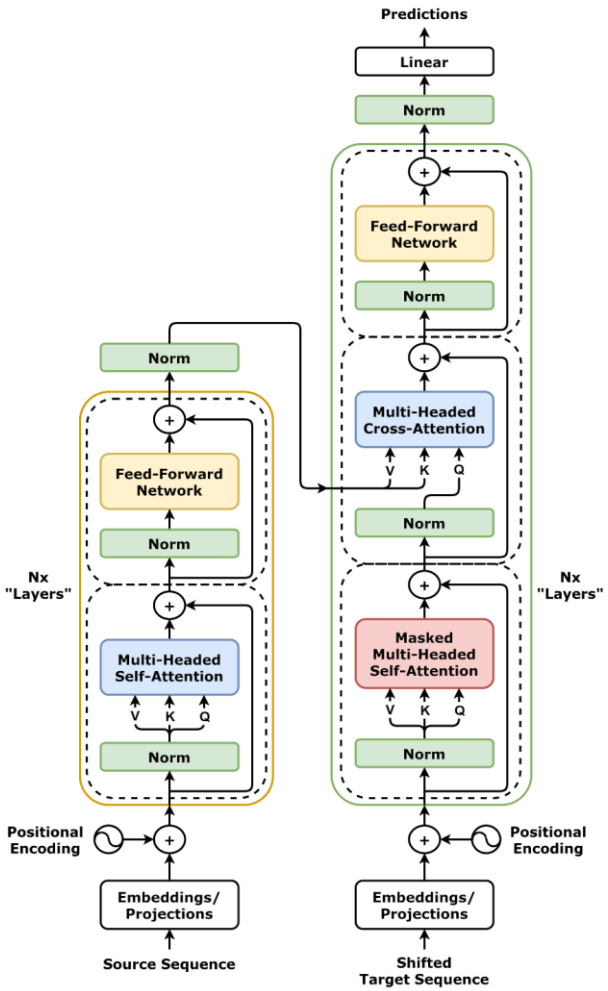
Hands-on session - Data-driven modelling of Lorenz System with ESN

- https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026/blob/main/04_ESN_Lorenz.ipynb





Very brief view on Transformers



Transformers:

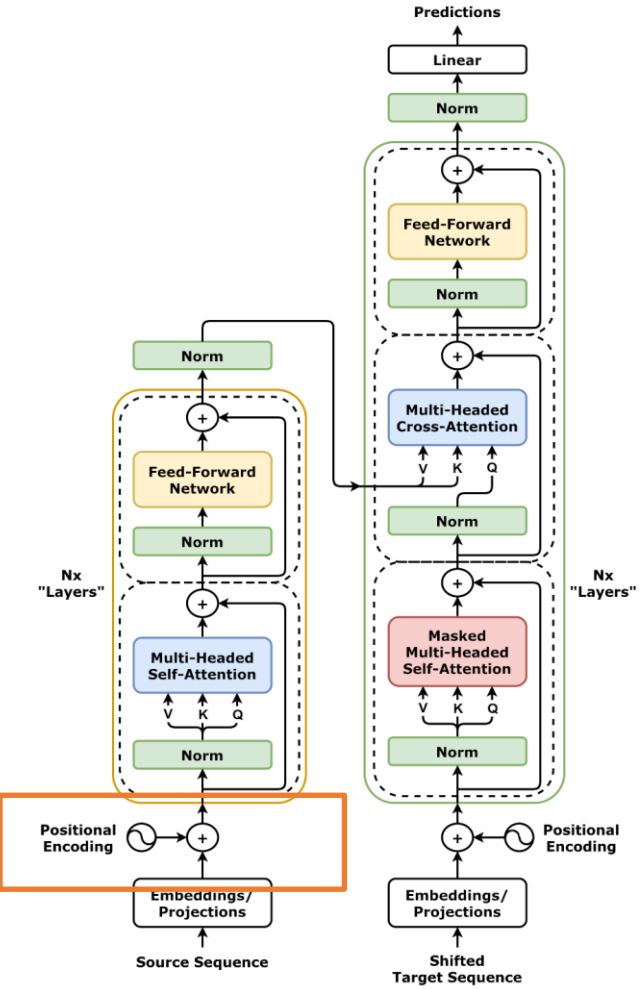
- Underpinning most recent Large Language Models

Key features:

- Does not rely on a “recurrence” mechanism
  - So potentially computationally faster
    - No back propagation through time
    - Parallelization possible
- Relies on “Positional Embedding” and “Attention mechanism”

[Attention is all you need, Vaswani et al., NIPS (30), 2017.]

Very brief view on Transformers



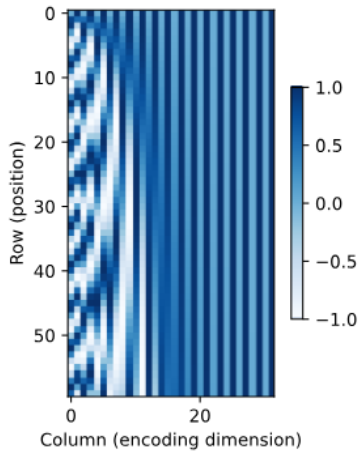
Encoder - Positional encoding

Assume a sequence $\boldsymbol{x}(n)$ of length N with $\boldsymbol{x} \in \mathbb{R}^d \rightarrow \boldsymbol{X} \in \mathbb{R}^{N \times d}$

Positional encoding matrix $\boldsymbol{P} \in \mathbb{R}^{N \times d}$ defined as:

$$P_{i,2j} = \sin\left(\frac{i}{M^{2j/d}}\right)$$

$$P_{i,2j+1} = \cos\left(\frac{i}{M^{2j/d}}\right)$$



Where M is a very large number ($M \gg N, d$).

The updated input with positional encoding is:

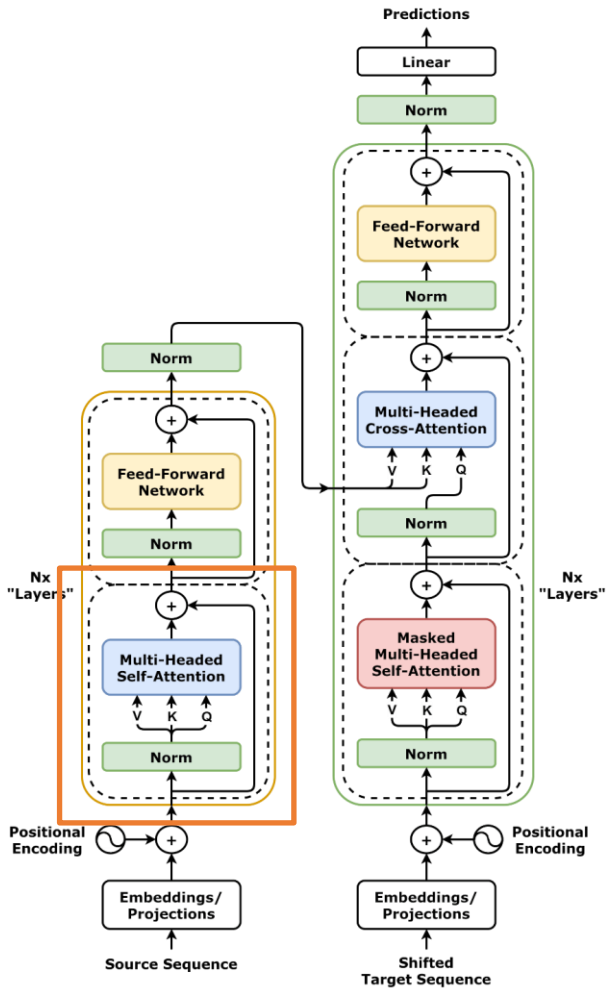
$$\boldsymbol{X} + \boldsymbol{P}$$

Characteristics of this positional encoding: Shift by offset δ equivalent to linear matrix multiplication:

$$\begin{pmatrix} P_{i+\delta,2j} \\ P_{i+\delta,2j+1} \end{pmatrix} = \begin{pmatrix} \cos(\delta\omega_j) & \sin(\delta\omega_j) \\ -\sin(\delta\omega_j) & \cos(\delta\omega_j) \end{pmatrix} \begin{pmatrix} P_{i,2j} \\ P_{i,2j+1} \end{pmatrix} \qquad \omega_j = \frac{1}{N^{2j/d}}$$

→ Provides mean to identify relative position of element in the sequence

Very brief view on Transformers



Encoder - Self-Attention mechanism

Assume a triplet of "(query, key, values)" $(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})$

- Query: what you're asking
- Keys: the way the underlying knowledge is indexed
- Values: the set of values associated with the keys

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$

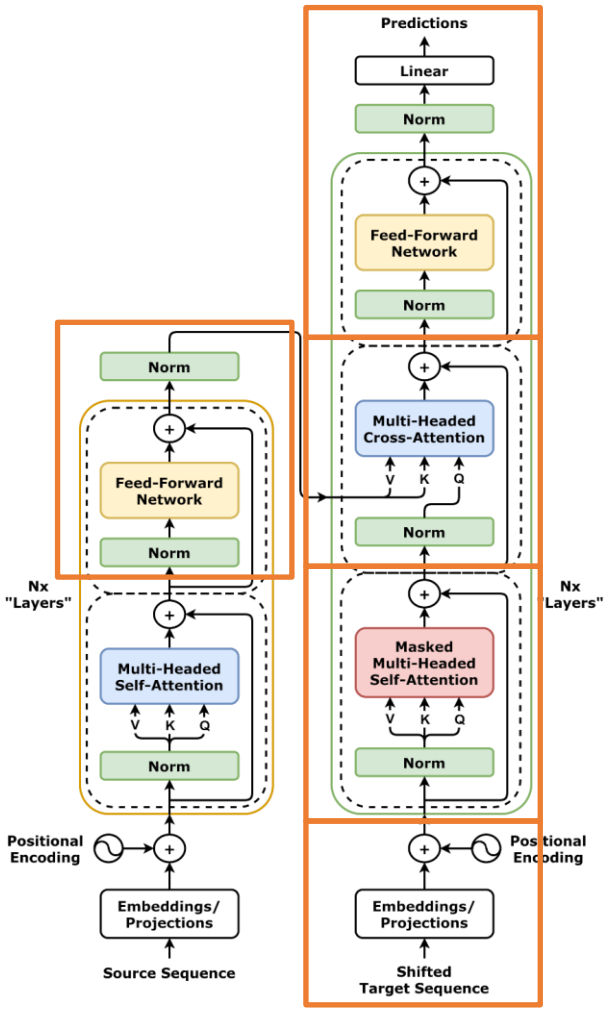
1. $\boldsymbol{Q}\boldsymbol{K}^T$: "Similarity of your query with respect to the keys"
   → Similar to a correlation between $\boldsymbol{Q}$ and $\boldsymbol{K}$
2. Softmax: provides a probability associated to the values
3. Multiplied by the value (to be predicted) $\boldsymbol{V}$

In practice for self-attention: $\boldsymbol{Q} = \boldsymbol{K} = \boldsymbol{V} \equiv \boldsymbol{X}$

All matrices multiplied by weight matrices (to be learned)

$$\text{multiheadattention} = \text{concat}\left[\text{Attention}\left(\boldsymbol{W}_q\boldsymbol{X}_q, \boldsymbol{W}_k\boldsymbol{X}_k, \boldsymbol{W}_v\boldsymbol{X}_v\right)\right]$$

Very brief view on Transformers



Encoder - Feedforward network

- "Combine" the predicted values and feed to cross-attention (in decoded)

Decoder – Positional Embedding/Self-Attention

- Positional embedding: applied on the target sequence
- Masked self-attention: similar as on the encoder-side but masking to ensure "causality" (i.e. no cross correlation with future attention)

Decoder – Cross-Attention

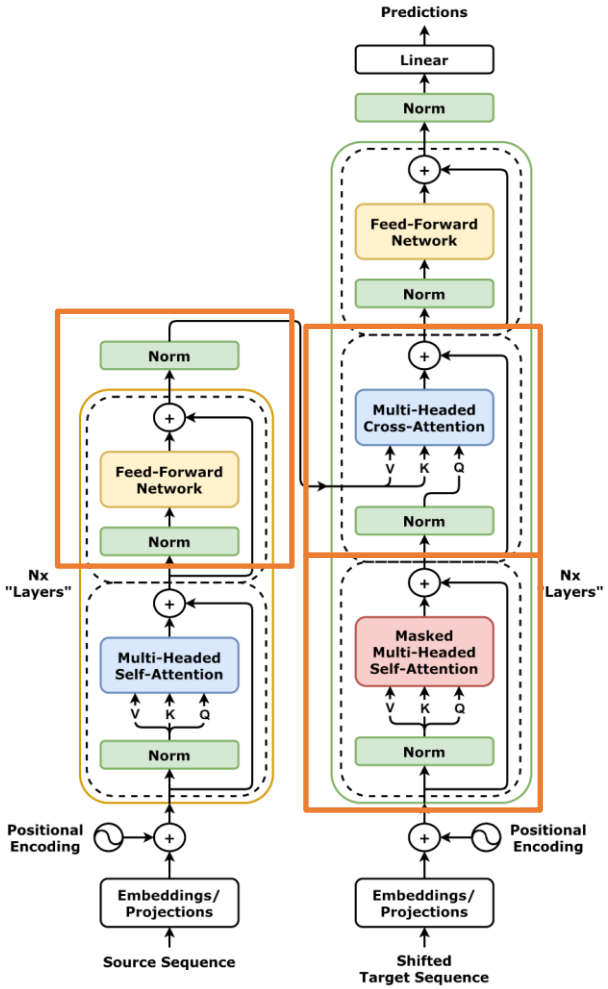
- Combine the self-attention (from target sequence) with transformed attention (from encoder)

$$\text{multiheadattention} = \text{concat}\left[\text{Attention}\left(\boldsymbol{W}_{\boldsymbol{q}}^{Dec}\boldsymbol{X}_{\boldsymbol{q}}^{Dec}, \boldsymbol{W}_{\boldsymbol{k}}^{Enc}\boldsymbol{X}_{\boldsymbol{k}}{}^{enc}, \boldsymbol{W}_{\boldsymbol{V}}^{Enc}\boldsymbol{X}_{\boldsymbol{v}}{}^{Enc}\right)\right]$$

Decoder – Feedforward/Linear

- Final transformation for prediction

Very brief view on Transformers

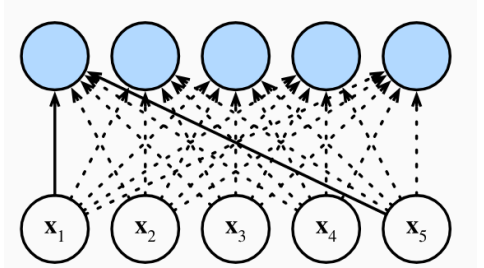


Core mechanism for “sequence handling”

- Attention-mechanism:

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$

→ $\boldsymbol{Q}\boldsymbol{K}^T$: parallelizable – does not require recurrence



Similar to a fully-connected NN approach

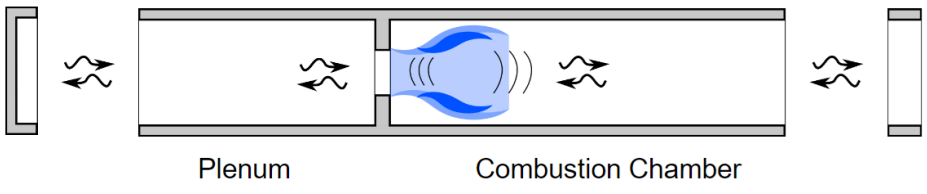
- Time-dependency: dependent on embedding length and dimension of $\boldsymbol{Q}$ (query length accepted)
- Can be very memory intensive
- But allow for potentially more efficient training through parallelization

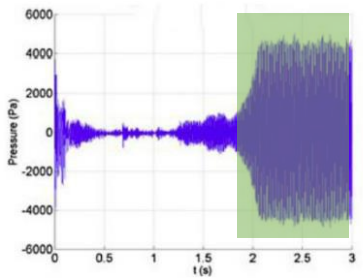
Outline

- Key concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and Dynamical Systems
  - System Identification for LTI system
  - Data-driven nonlinear dynamical system modelling
- **Applications to reacting flows**
  - Flame dynamics modelling
  - Reduced-order modelling

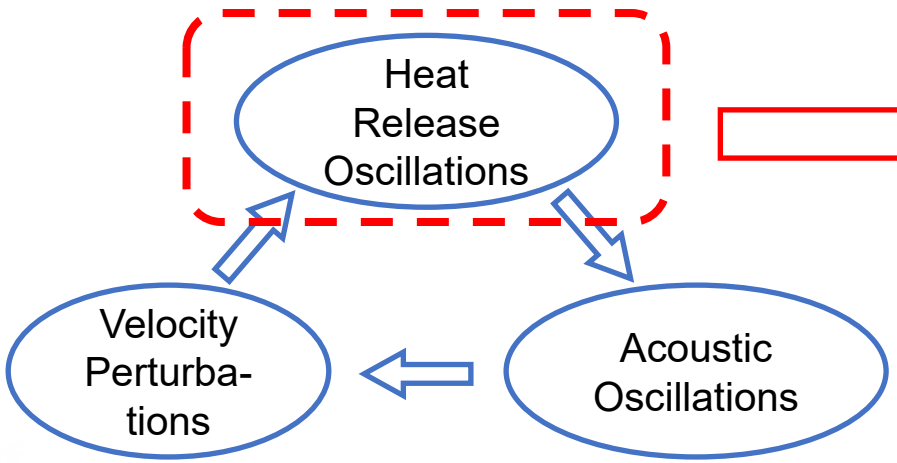
Typical dynamical modelling in turbulent (reacting) flows: Flow dynamics modelling

- Thermoacoustic instabilities: High amplitude pressure oscillations in premixed combustors

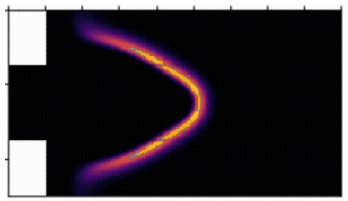




Thermoacoustic instabilities: result from coupling between HRR and acoustic fluctuations

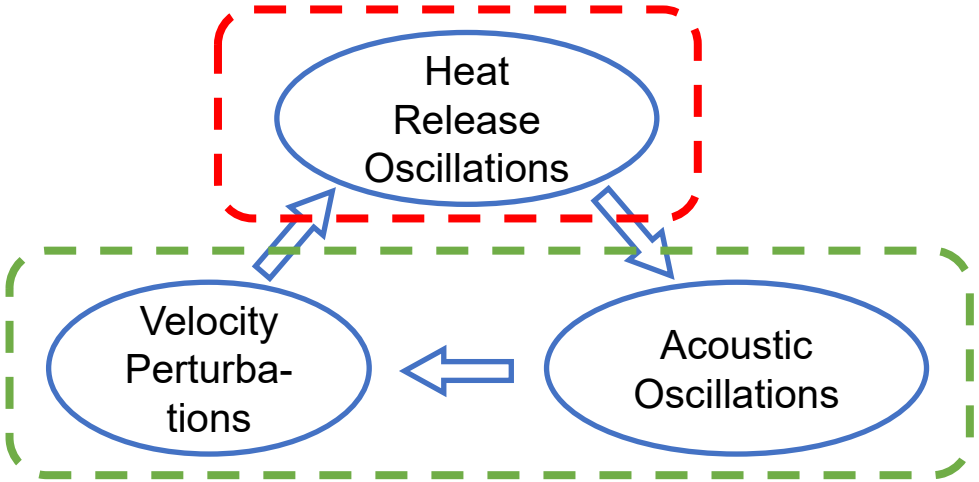


$$\dot{\boldsymbol{\phi}} = \boldsymbol{N}(\boldsymbol{\phi}, \boldsymbol{p}, \boldsymbol{F})$$



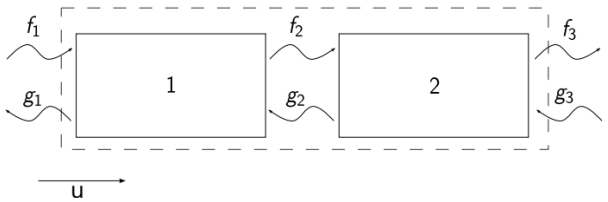
$$\rightarrow \dot{\boldsymbol{Q}} = \mathcal{F}(\boldsymbol{\phi}, \boldsymbol{F})$$

Divide and Conquer approach relies on interacting flame and acoustic models



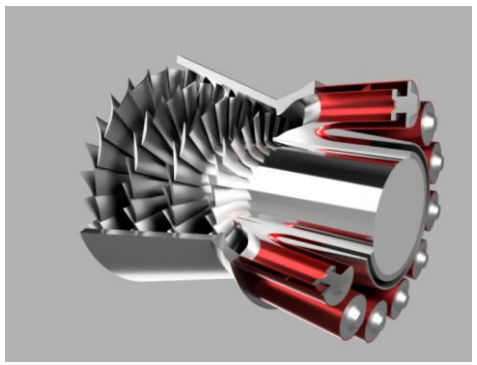
Flame model: $\dot{Q}' = \mathcal{F}(u')$

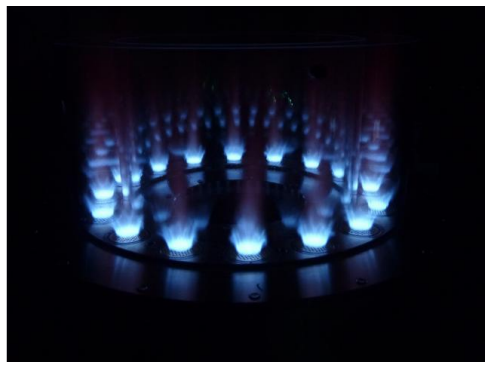
Acoustic network model

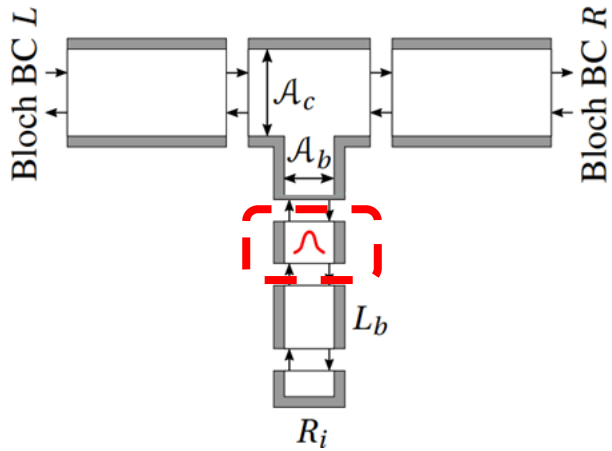


$$\dot{x_1} = A_1 x_1 + B_1 \begin{bmatrix} f_1 \\ g_2 \end{bmatrix} \quad y_1 = \begin{bmatrix} g_1 \\ f_2 \end{bmatrix}$$

$$\dot{x_2} = A_2 x_2 + B_2 \begin{bmatrix} f_2 \\ g_3 \end{bmatrix} \quad y_2 = \begin{bmatrix} g_2 \\ f_3 \end{bmatrix}$$





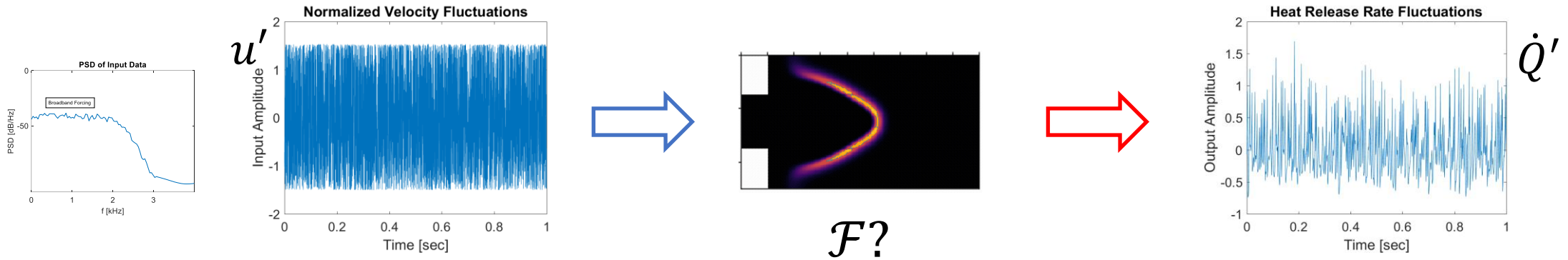


Essential of the flame dynamics can be obtained with single broadband forcing simulation

- What data do we need to model the flame dynamics?

$$\dot{Q}' = \mathcal{F}(u')$$

- From system identification:
  - Dataset with broadband forcing contains all the necessary information

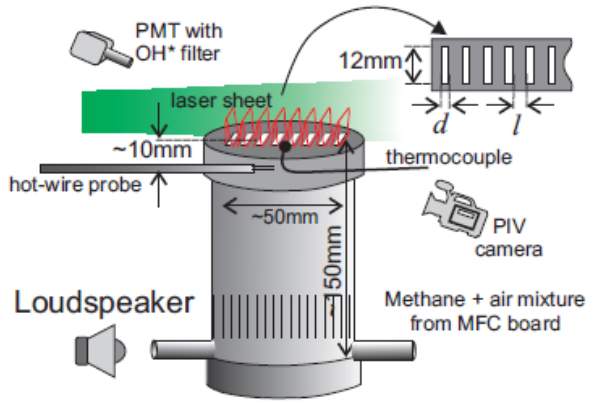


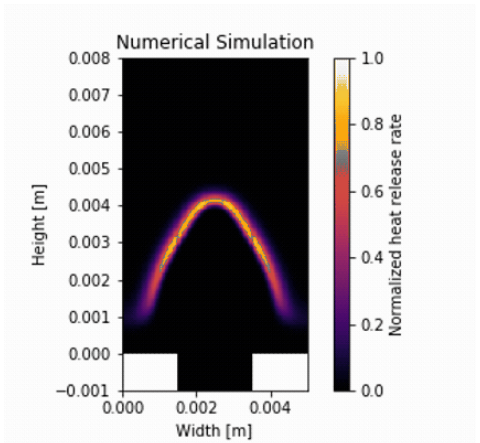
- How to model this?

  → Typical supervised learning problem applied to sequential data

Test case is a laminar premixed methane/air flame

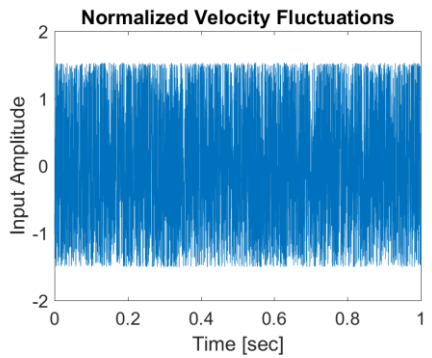
- Fully resolved lean premixed methane-air slit burner at $\phi = 0.8$



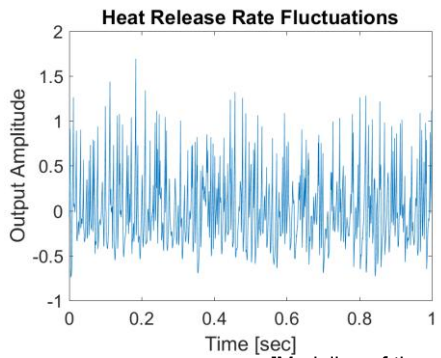


- Dataset: One simulation broadband excitation at high amplitude

$u'(t)$



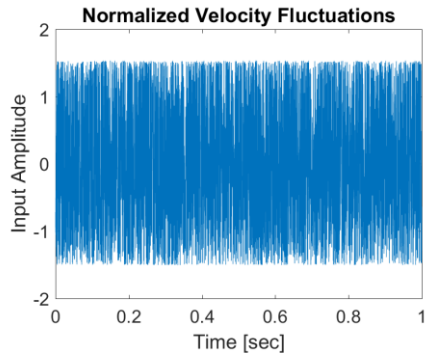
$\dot{Q}'(t)$



[Modeling of the nonlinear flame response of a Bunsen-type flame via multi-layer perceptron, Tathawadekar et al., Proc. Comb. Inst. (38), 2021]

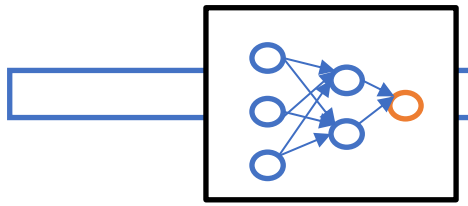
Neural networks can accurately learn nonlinear flame dynamics

NN trained with one broadband simulation at high amplitude

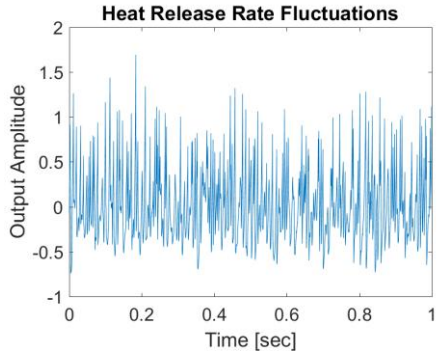


$$\begin{bmatrix} u'(t) \\ u'(t-\Delta t) \\ \vdots \\ u'(t-n\Delta t) \end{bmatrix}$$

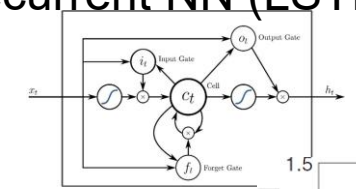
Feedforward NN

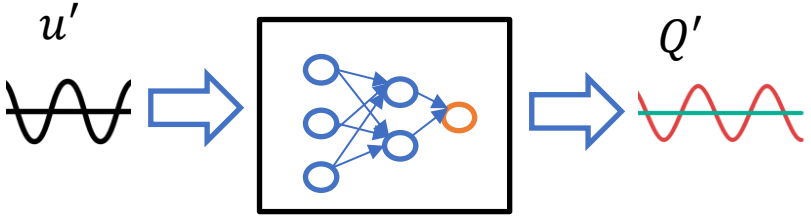


$[\dot{Q}'(t)]$

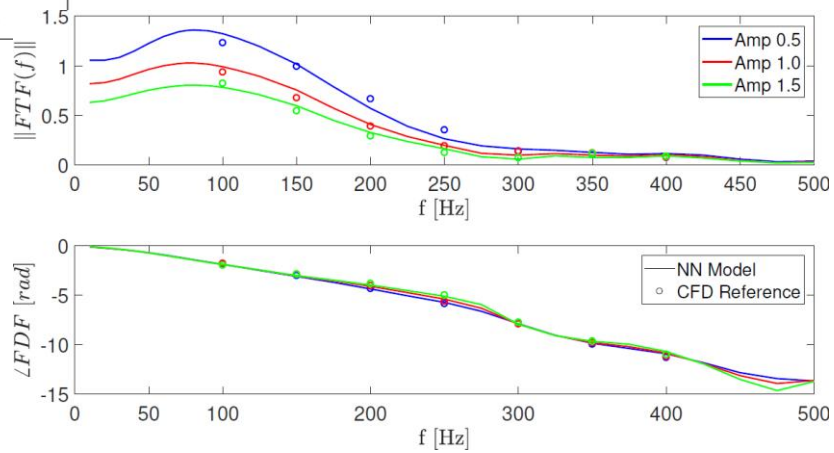


Recurrent NN (LSTM)





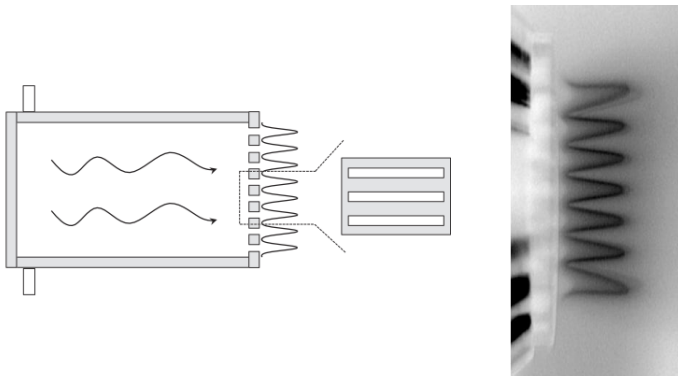
Frequency response of trained network

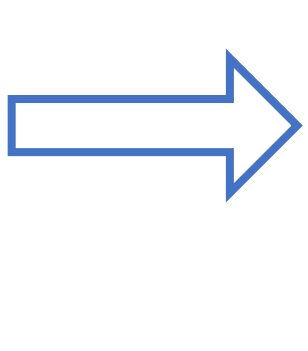


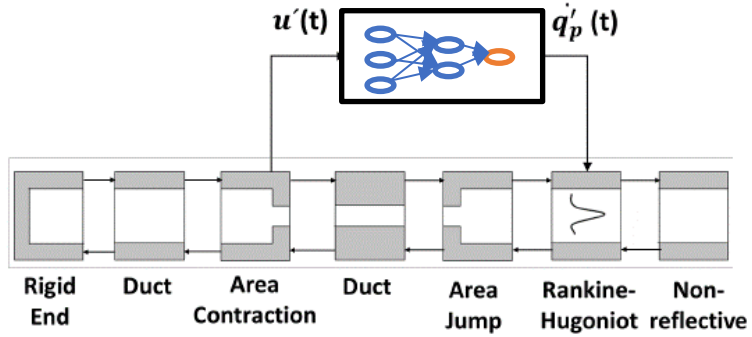
[Modeling of the nonlinear flame response of a Bunsen-type flame via multi-layer perceptron, Tathawadekar et al., Proc. Comb. Inst. (38), 2021]
[Physics-informed recurrent neural networks for linear and nonlinear flame dynamics, Yadav et al., Proc. Comb. Inst. (39), 2023]

Integrated NN and acoustic network predicts accurately the thermoacoustic instability

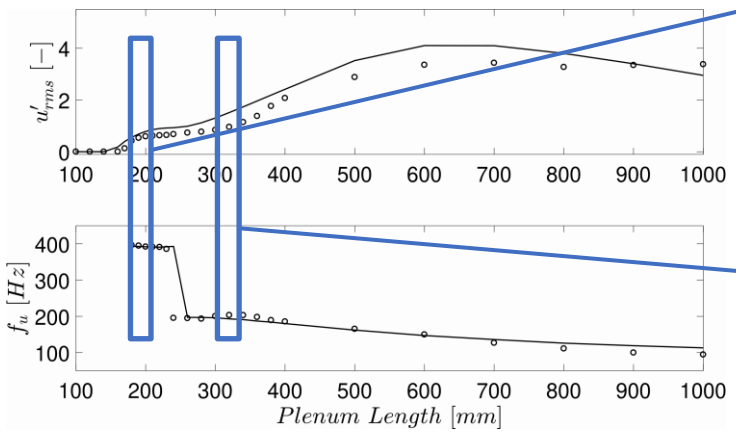
- Trained NN with acoustic model describes the multi-slit combustor

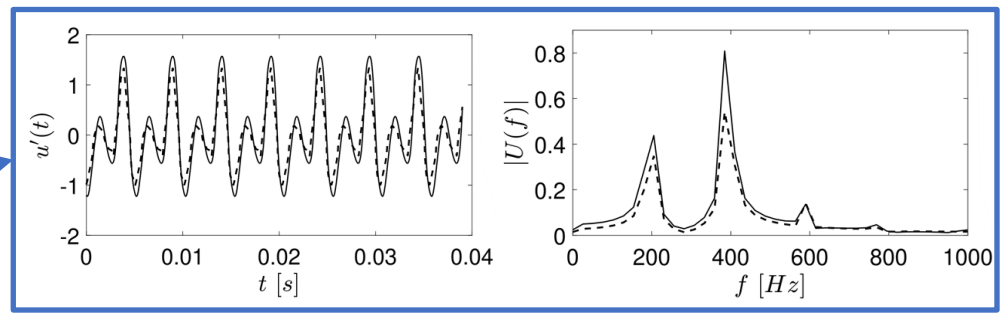


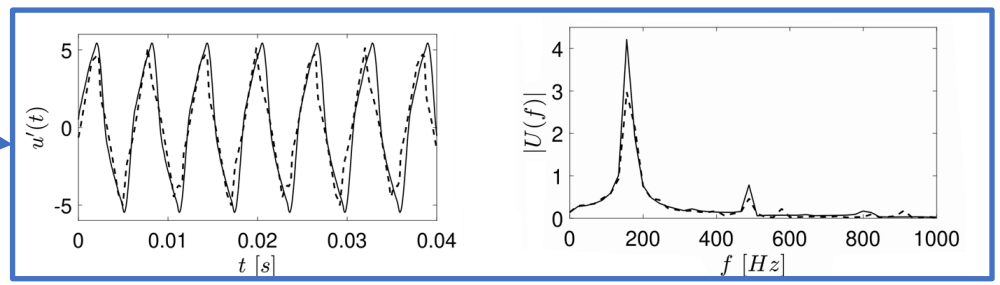




Design stability analysis



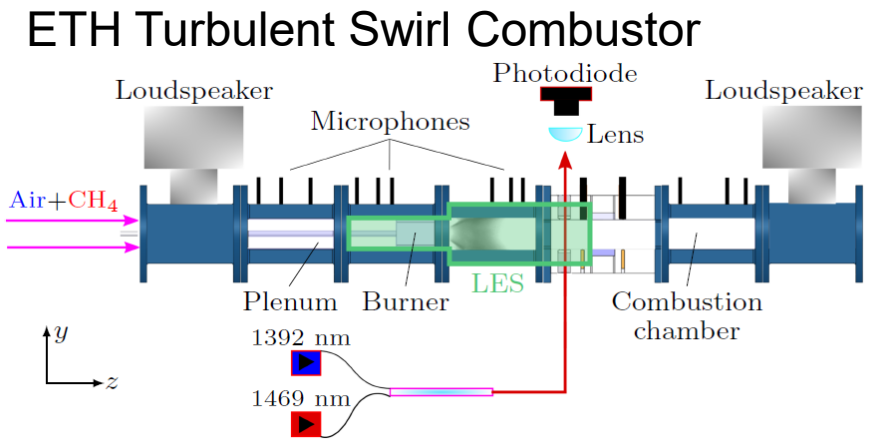


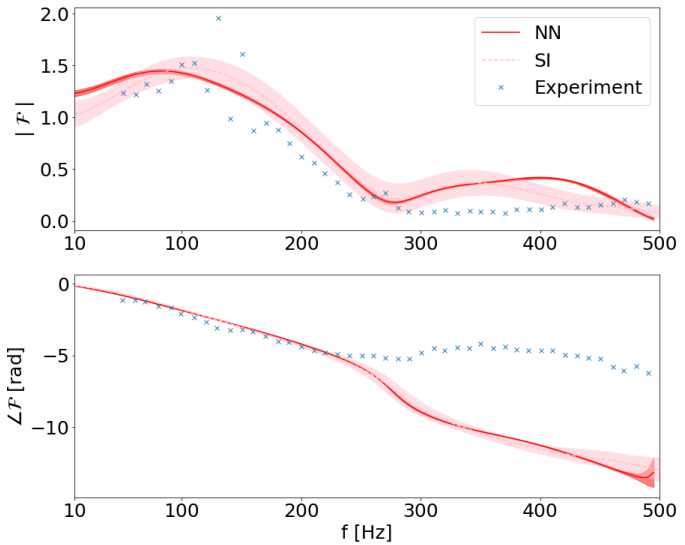


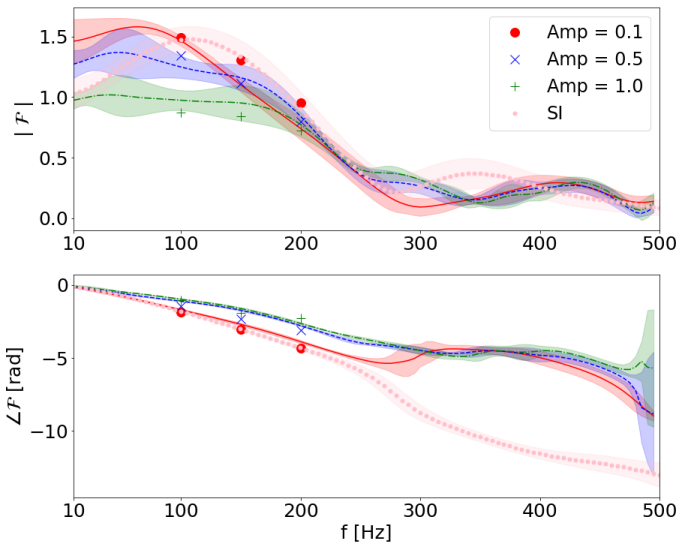
[Modeling of the nonlinear flame response of a Bunsen-type flame via multi-layer perceptron, Tathawadekar et al., Proc. Comb. Inst. (38), 2021]

Extension to turbulent flames

- Previous methodology also applicable to turbulent flames







Increased challenge due to turbulent fluctuations

[Linear and nonlinear flame response prediction of turbulent flames using neural network models Tathawadekar et al., Int. J. Spray Combust. Dyn. (16), 2024]

Hands-on session – Modelling of nonlinear flame dynamics of a laminar slit flame

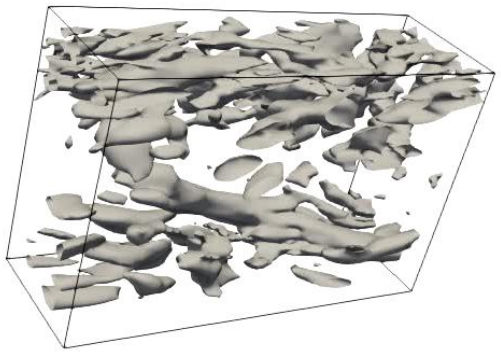
- https://github.com/ndoan-icl/CYPHER_MLSchool_Brussels2026/blob/main/05_FlameDynamics.ipynb
- See other set of slides for exercises steps

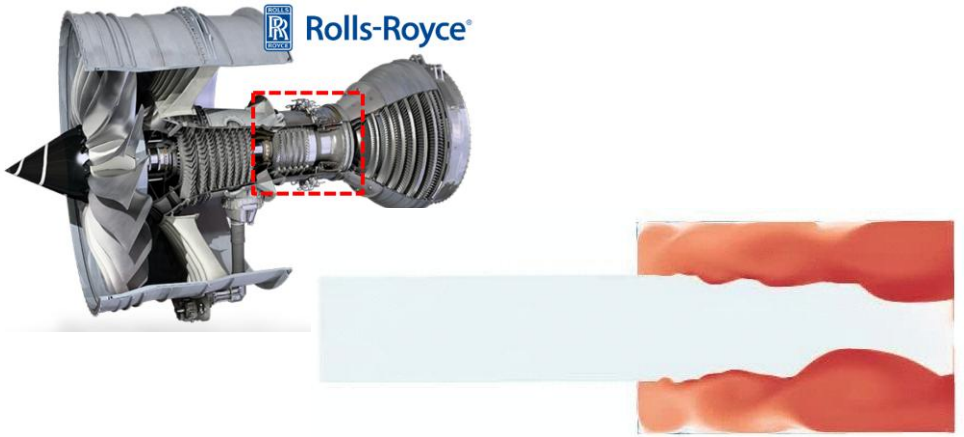
Outline

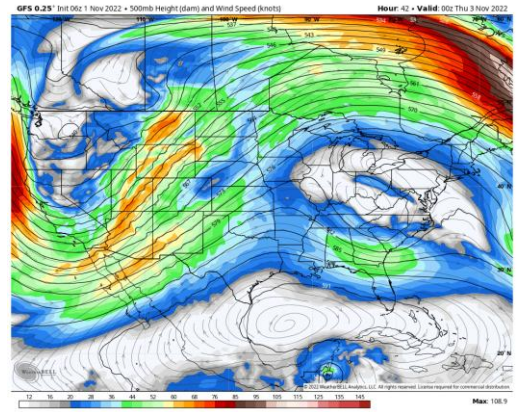
- Key concepts in Dynamical Systems and Chaos
- Machine Learning for Dynamical Systems
  - Sequential data and Dynamical Systems
  - System Identification for LTI system
  - Data-driven nonlinear dynamical system modelling
- **Applications to reacting flows**
  - Flame dynamics modelling
  - **Reduced-order modelling**

Learning turbulent dynamics of high-dimensional flows

- Most turbulent flows are highly dimensional







How can we handle those with all the methods seen so far?

→ Split the tasks

Spatial structures
→ Handled with CNN
(form of nonlinear modal decomposition)

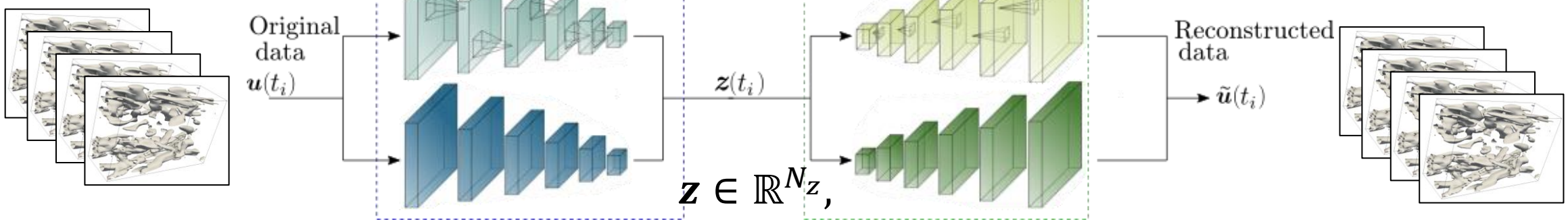
Temporal chaotic dynamics
→ Handled with RNN

→ In essence similar to a POD-Galerkin ROM approach

The spatial structures can be identified with CNN-based networks

- CNN-based autoencoder: discovers efficient latent space
  - Based on identifying “spatial structures”
  - Encoder: finds an efficient reduced order representation (latent space)
  - Decoder: reconstructs the full flow state from the latent space

$\boldsymbol{u} \in \mathbb{R}^{N_u}, N_u \sim 10^5$



$\tilde{\boldsymbol{u}} \approx \boldsymbol{u}$

$\boldsymbol{z} \in \mathbb{R}^{N_z}$,

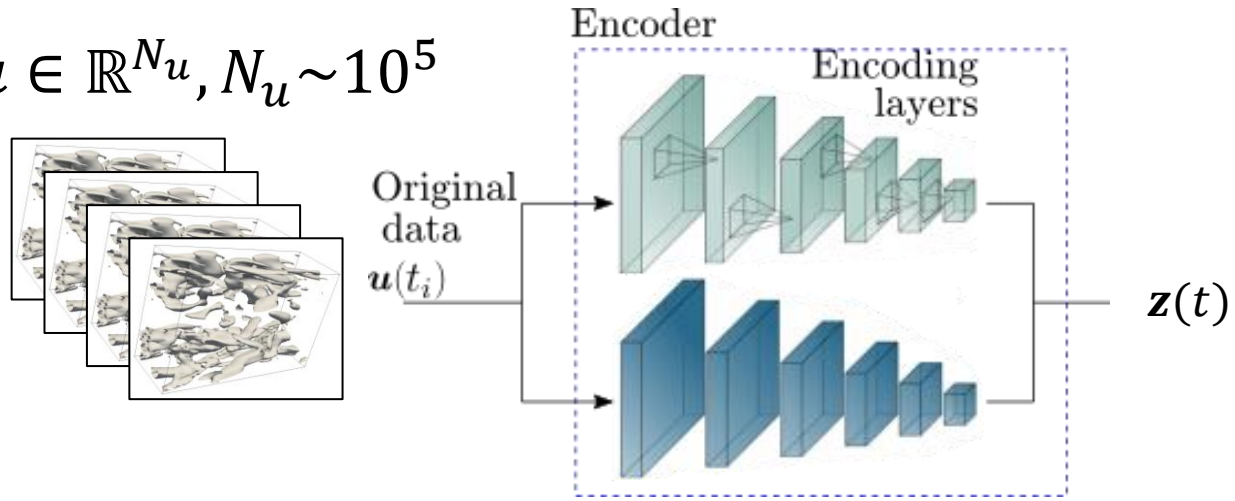
$N_z \ll N_u$

Trained by minimizing: $J = \frac{1}{N_t} \sum_i \left\| \boldsymbol{u}^{(i)} - \tilde{\boldsymbol{u}}^{(i)} \right\|^2$

[Predicting turbulent dynamics with the convolutional autoencoder echo state network, Racca et al., J. Fluid Mech. (975), 2023]
[Convolutional autoencoded echo state network for the prediction of extreme events in turbulence, Doan et al., CTRSP-2022, 2022]

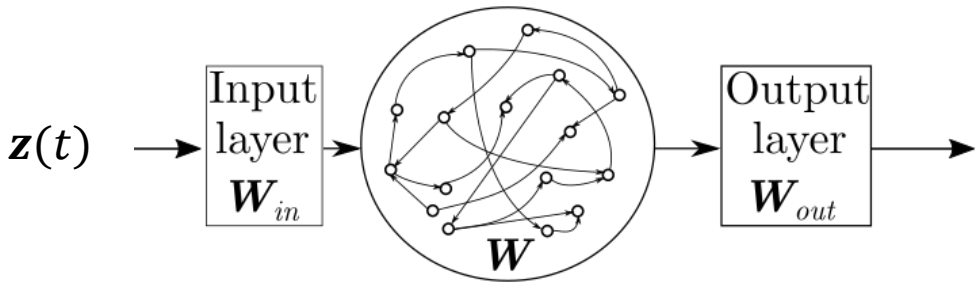
The encoder provides flow dynamics in the latent space

$\boldsymbol{u} \in \mathbb{R}^{N_u}, N_u \sim 10^5$



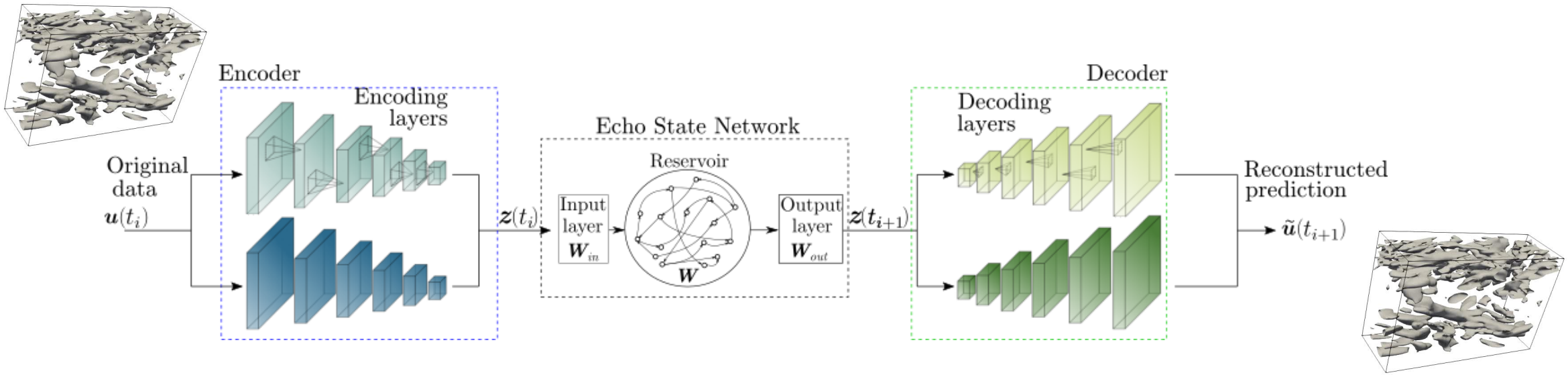
- "Encoded" version of original dataset
- Can be used to learn the dynamics
- Decoder can be used to recover the full physical state

We can learn that dynamics (in the latent space) with an RNN



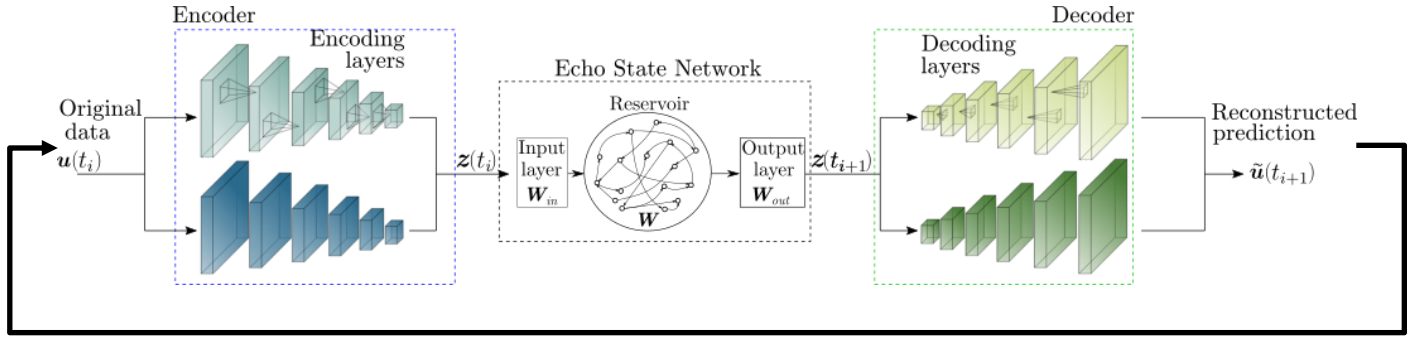
By combining CNN and RNN, we can handle high-dimensional flows

- Overall architecture:

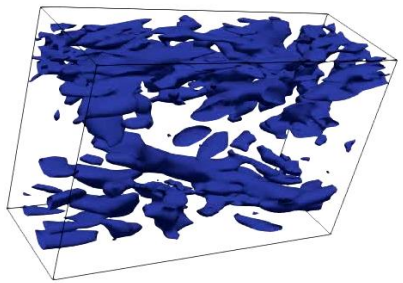


The hybrid model can accurately reproduce the flow dynamics

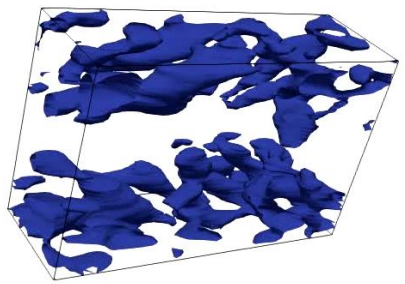
After training



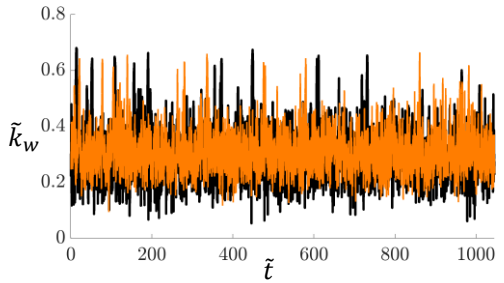
Actual

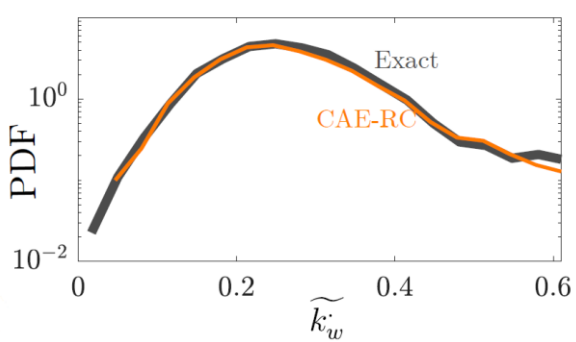


ML model



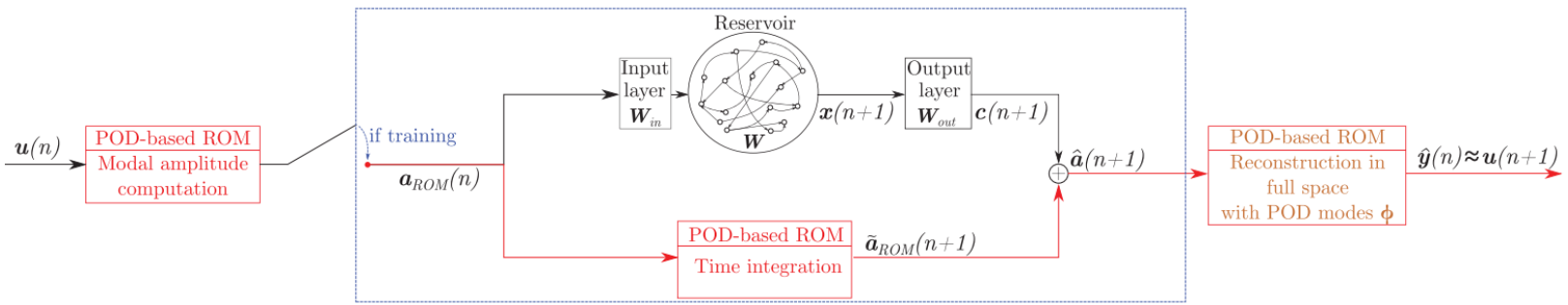
Extreme events statistics





Summary

- Chaotic systems are difficult to predict
  - Comparison based on: statistics/attractors and short-term predictions
- Sequential data requires particular processing
- Modelling of dynamical system
  - If LTI system: Impulse response describes the entire dynamics
  - If nonlinear:
    - Families of Recurrent Neural Networks exists
- Note: Combination of methods possible



[Chaotic systems learning with hybrid echo state network/proper orthogonal decomposition based model, Lesjak & Doan, Data-Centric Eng (2), 2021.]

Questions?

n.doan@imperial.ac.uk